

"Resumen" Redes y Comunicaciones

Clase 1

¿Qué es una red de computadoras/ordenadores?

- Análisis del punto de vista sistémico:
 - Red de computadoras: Grupo de computadoras/dispositivos interconectados
 - Objetivo: Compartir recursos (dispositivos, información, servicios).
 - El conjunto de computadoras, software de red, medios y dispositivos de interconexión forman un sistema de comunicación
 - Ejemplos: red de sala de PCs, red Universitaria, Internet

Componentes de un sistema de comunicación

- Fuente (Software).
- Emisor/Transmisor (Hardware).
- Medio de transmisión y dispositivos intermedios (Hardware).
- Procesos intermedios que tratan la información (Software y Hardware).
- Receptor (Hardware).
- Destino (Software).
- Otros: Protocolos (Software), Información, mensaje transmitido (Software).
- Señal de Información, materialización del mensaje sobre el medio (Hardware?).

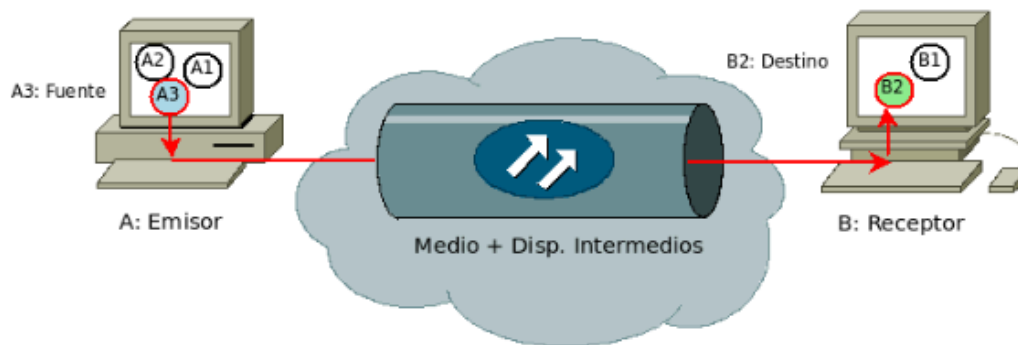


Figure: Sistema de Comunicaciones.

Componentes de una red

- Fuera del punto de vista sistémico podemos ver un gran número de componentes:
 - Computadoras, en el modelo de Internet: Hosts (PCs, laptops, servidores)
 - Routers/switches, Gateways, AP (Access points)
 - NIC (placas de red), Modemos.
 - Vínculos/enlaces: conformados por:
 - Medios: cables, fibras ópticas, señales electromagnéticas, antenas, interfaces, etc.
 - Programas: Browsers, Servidores Web, Clientes de Mail, Servidores de Streaming
 - Etc
- Las componentes de la red deben interactuar y combinarse a través de reglas.

Protocolos

Protocolos

Protocolo

Es el conjunto de conductas y normas a conocer, respetar y cumplir no sólo en el medio oficial ya establecido, sino también el

social y laboral.

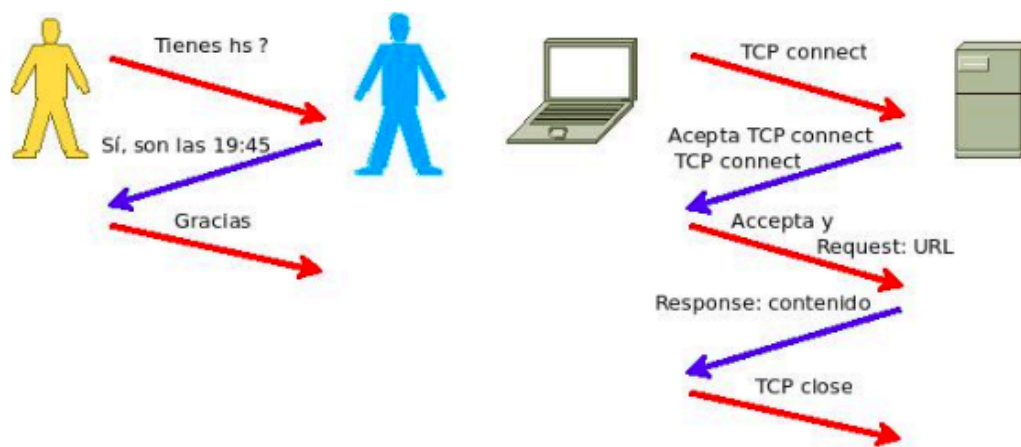


Figure: Protocolo

Nota: La imagen compara la comunicación humana con la de los dispositivos, mostrando que ambos necesitan reglas y secuencias ordenadas (protocolos) para entenderse.

Un protocolo define el formato, orden de los mensajes intercambiados y las acciones que se llevan a cabo en la transmisión y/o recepción de un mensaje u otro evento.

Protocolo de Red

Conjunto de reglas que especifican el intercambio de datos u órdenes durante la comunicación entre las entidades que forman parte de una red. Permiten la comunicación y están implementados en las componentes.

Protocolos de Redes propietarios

- A principios de los 80', las compañías comenzaron a implementar redes propias (privadas y cerradas).
- Primeras Redes Propietarias.
- Consecuencia: Cada red tenía sus especificaciones propias (protocolos).

- Resultados: Incompatibilidad. La comunicación entre redes era muy difícil, evolución más lenta, carencia de estándares.
- Complejidad de modelos.

Combinación de protocolos

- La cantidad de componentes de red a interactuar genera complejidad, se requiere una organización de las mismas.
- Se requieren Modelos de Organización.
- Modelo en Capas: Layering, divide la complejidad en componentes reusables.
 - Reduce complejidad en componente más pequeñas.
 - Las capas de abajo ocultan la complejidad a las de arriba, abstracción.
 - Las capas de arriba utilizan servicios de las de abajo: Interfaces, similar a APIs.
 - Los cambios en una capa no deberían afectar a las demás si la interfaz se mantiene.
 - Facilita el desarrollo, evolución de las componentes de red asegurando interoperabilidad.
 - Facilita aprendizaje, diseño y administración de las redes

Modelo OSI

Modelo TCP-IP

- Modelo que se convirtió en estándar.
- ¿Qué protocolos se encuentran en Internet?
 - Modelo Abierto.
 - Varios protocolos de nivel de enlace: Ethernet, t, 802.3, PPP, HDLC, Frame-Relay, 802.11a/b/g/n/ac, MPLS, DOCSIS, GPON, etc (No definidos por TCP/IP).
 - Protocolos propios de Internet y Transporte (Núcleo): ARP, IP, ICMP, TCP, UDP, OSPF, BGP, etc.
 - Protocolos de Aplicaciones: DNS, HTTP, FTP, SSH, SMTP, IMAP, etc.

- API abierta para generar nuevos protocolos.
- Modelo de 5 (cinco) capas:
 - Capa de aplicación(Process/Application)
 - Capa de transporte o Host-to-host.
 - Capa de Internet o Internetworking.
 - Capa de Enlace(Link Layer).
 - Capa Física
- Por simplicidad algunos autores hablan de 4 capas, agrupando la capa de enlace y la capa física en una sola capa que llaman capa de acceso a la Red.

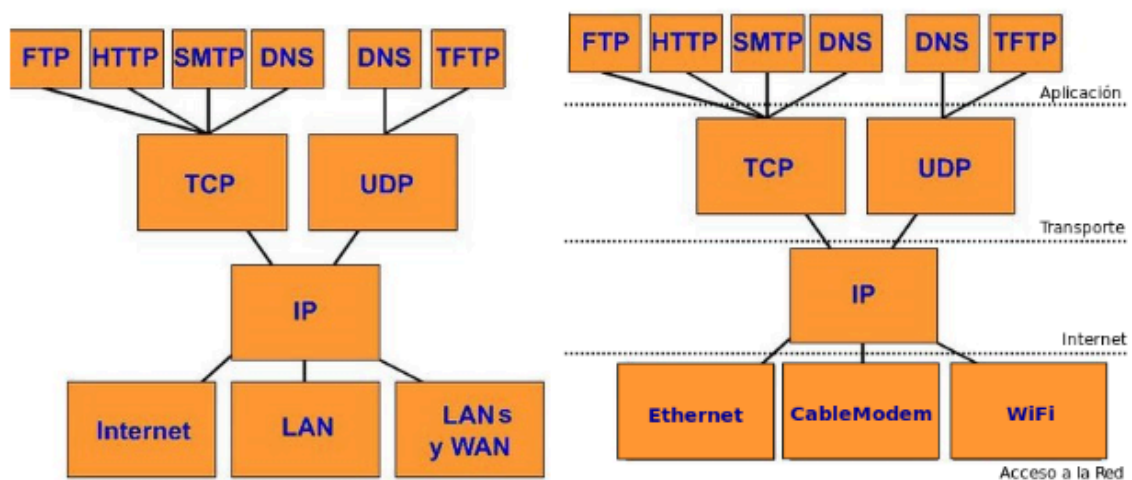


Figure: Modelo TCP/IP

Comparación OSI vs TCP-IP

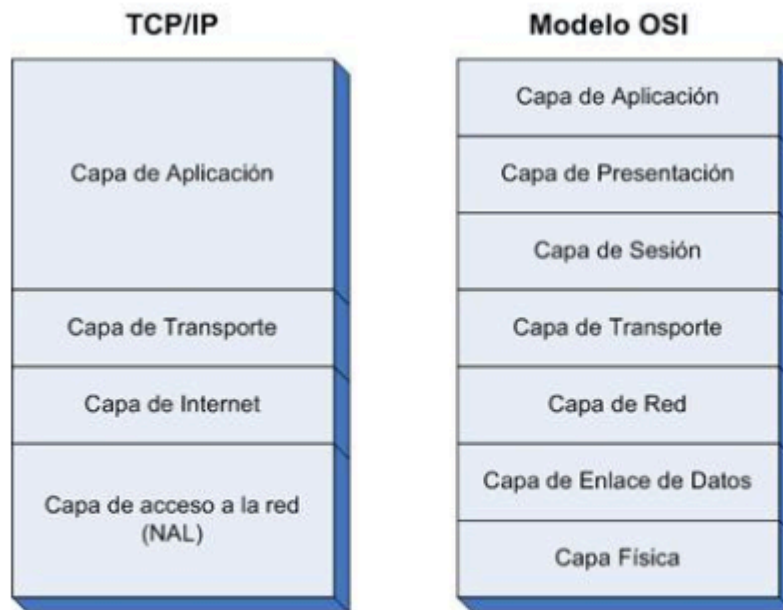


Figure: Modelo OSI vs TCP/IP

- Similitudes:
 - Ambos se dividen en capas.
 - Ambos tienen capas de aplicación, aunque incluyen servicios distintos.
 - Ambos tienen capas de transporte similares.
 - Ambos tienen capa de red similar pero con distinto nombre.
 - Se supone que la tecnología es de conmutación de paquetes (no de conmutación de circuitos).
 - Es importante conocer ambos modelos.
- Diferencias
 - TCP/IP combina las funciones de la capa de presentación y de sesión en la capa de aplicación.
 - TCP/IP combina la capas de enlace de datos y la capa física del modelo OSI en una sola capa.
 - TCP/IP más simple porque tiene menos capas.
 - Los protocolos TCP/IP son los estándares en torno a los cuales se desarrolló Internet, de modo que la credibilidad del modelo
 - TCP/IP se debe en gran parte a sus protocolos.

- El modelo OSI es un modelo “más” de referencia, teórico, aunque hay implementaciones.

Encapsulamiento

- Cada capa define su PDU: Protocol Data Unit

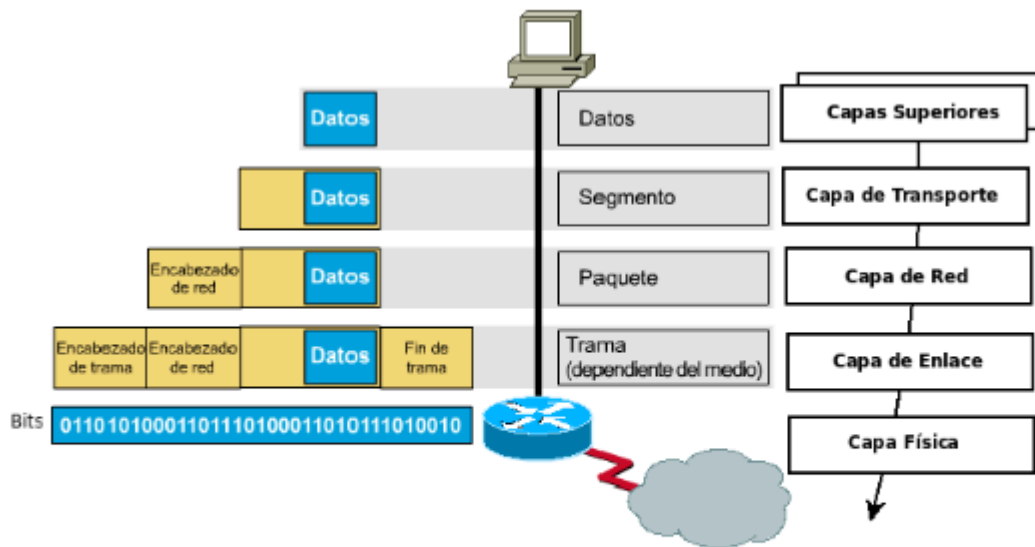


Figure: Encapsulamiento y PDUs

A medida que los datos descienden por las capas, sufren un proceso de encapsulamiento: cada capa envuelve los datos en una PDU más grande agregándole sus propios encabezados de control

Dispositivos y capas

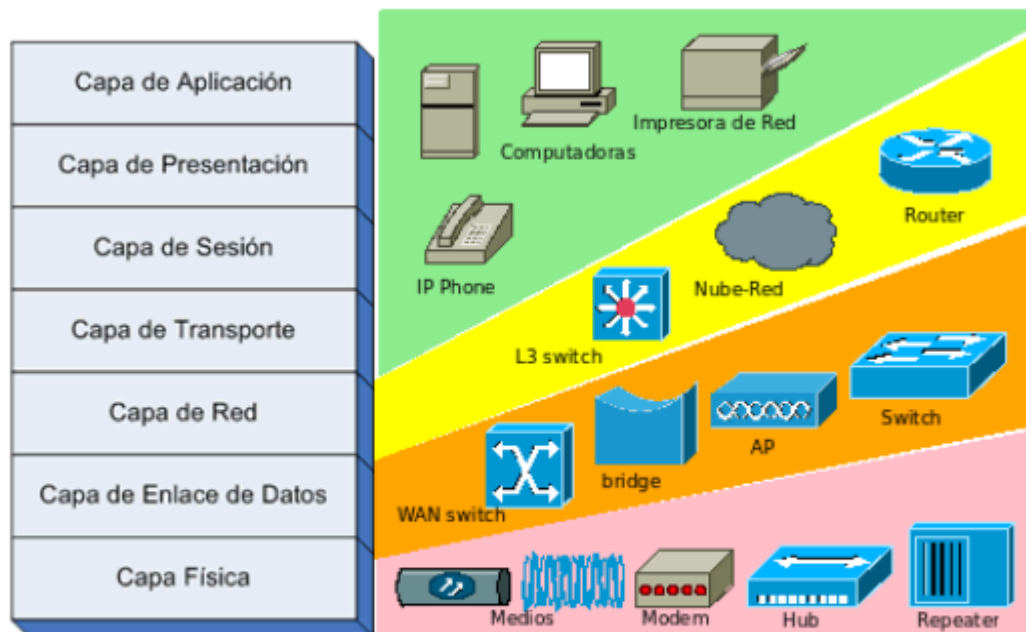


Figure: Dispositivos y Capas

Comunicación entre capas Peer-Peer

- Cada capa usa el servicio de la de abajo.
- Cada capa se comunica con la capa del otro extremo.

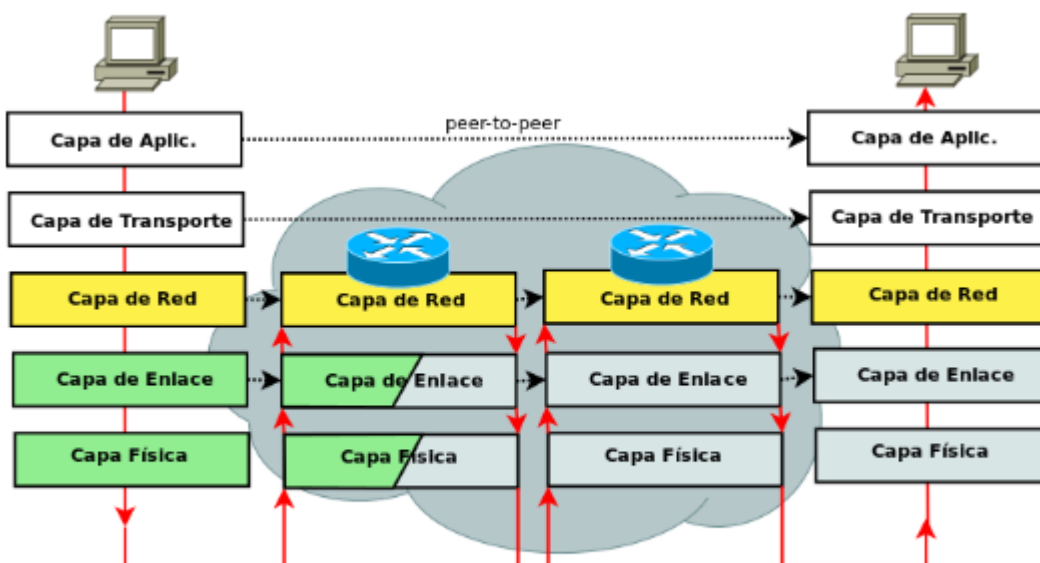


Figure: Comunicación Peer-Peer

Clasificación de Redes

Clasificación de Redes

- Diferentes clasificaciones de acuerdo a diferentes aspectos.
- Se pueden mencionar:
 - Clasificación por cobertura, distancia, alcance.
 - Clasificación por acceso abierto o privado
 - Clasificación por topología física
 - Clasificación por tipo de conexión/medio
 - Etc

Clasificación por cobertura

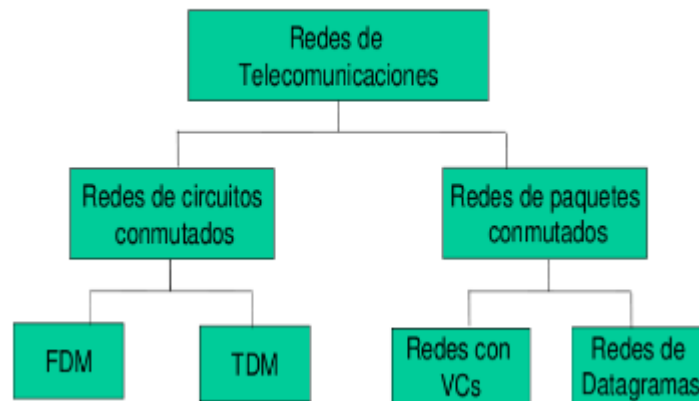
- LAN: (Local Area Network). Red de cobertura local. Ethernet, Wi-Fi.
- MAN: (Metropolitan Area Network). red de cobertura metropolitana, dentro de una ciudad. MetroEthernet, MPLS, Wi-Max.
- WAN: (Wide Area Network). red de cobertura de área amplia. Geográficamente distribuida. PPP, Frame-Relay, MPLS, HDLC, SONET/SDH.
- SAN: (Storage Area Network). red de almacenamiento. iSCSI, Fibre Channel, ESCON.
- PAN: red de cobertura personal. Red con alcance de escasos metros para conectar dispositivos cercanos a un individuo. Bluetooth, IrDA, USB.
- Otros términos: CAN (Controller Area Network o Campus Area Network), NAN (Near-me AN, NFC), ...

Clasificación Públicas y Privadas

- **Internet:** red pública global, tecnología TCP/IP.
- **Intranet:** red privada que utiliza la tecnología de Internet.
- **Extranet:** red privada virtualizada sobre enlaces WAN: Internet. Intranet con acceso de usuarios remotos. VPN (Virtual Private Network) IPSec, PPTP, SSL, OpenVPN, L2TP. Una intranet mapeada sobre una red pública como Internet

Clasificación Física de redes

- Redes de Conmutación de Circuitos.
- Redes de Conmutación de Tramas/Paquetes.
 - Servicios Orientados a Conexión. Circuitos Virtuales.
 - Servicios NO Orientados a Conexión. Datagramas.



Internet

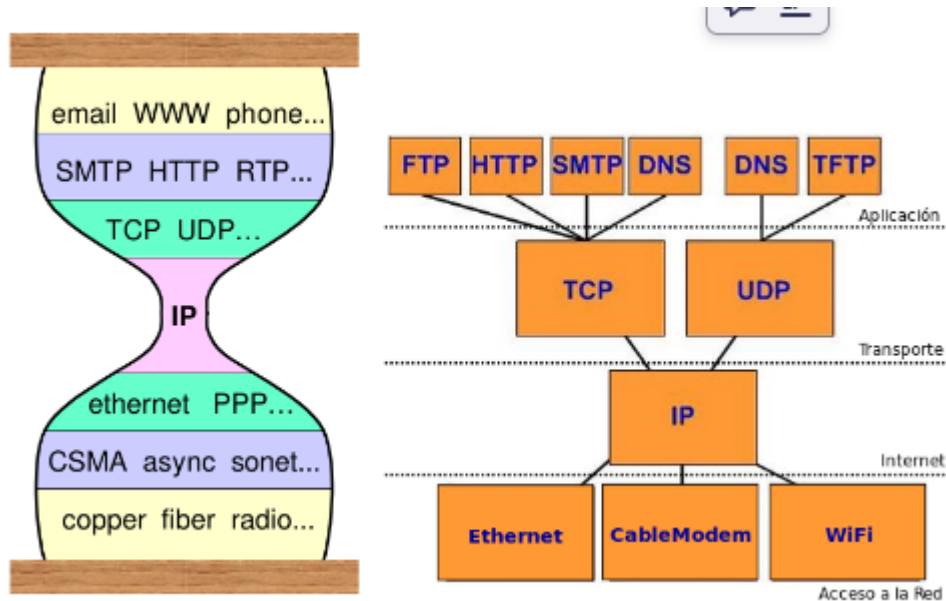
Internet

¿Qué es Internet?

- Es una red de redes de computadoras, descentralizada, pública, que ejecutan el conjunto abierto de protocolos (suite) TCP/IP. Integra diferentes protocolos de un nivel más bajo:
 - Interetworking

Modelo de Internet

Modelo de forma de reloj de arena (hourglass):



Modelo simplificado de Internet

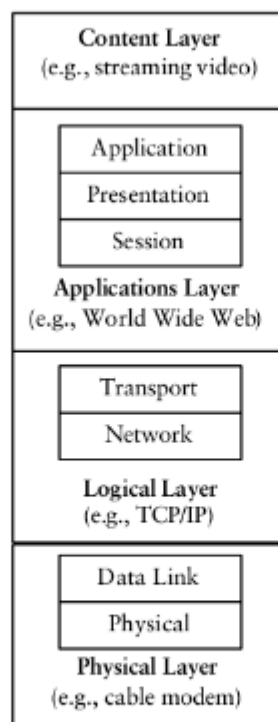


Figure: Modelo Simplificado de Internet de 4 capas

Objetivos/Historia

- Inicios de 1960': Red militar para la guerra fría ?? (aún no existía TCP/IP). Packet Switching Theory: paper de Kleinrock, usar

paquetes en lugar de circuitos en ARPANET: RAND Corp, Leonard Kleinrock del MIT trabajan sobre la red, BBN implementa IMPs.

- Primera vez On-Line 1969: Conectaba las Universidades: Stanford (SRI), Utah, UCLA, UCSB (UC Santa Barbara).
- Nuevo protocolo LAN 1973: Ethernet, Bob Metcalfe en Xerox PARC.
- Cambio a TCP/IP 1983: Desde NCP a TCP/IP. Vinton Cerf y Robert E. Kahn.
- Luego, NSFNET 1985: Red Científica e Investigación, Usada por las Universidades.
- Continuando 1988: Comienza como negocio, nuevas oportunidades.
- Hoy 2020: Tele-trabajo, Clases Virtuales, Redes Sociales,

Request For comments

Son los documentos técnicos oficiales, elaborados por la comunidad de ingenieros de Internet (la IETF), que describen cómo deben funcionar los protocolos (IP, TCP, HTTP, etc.).

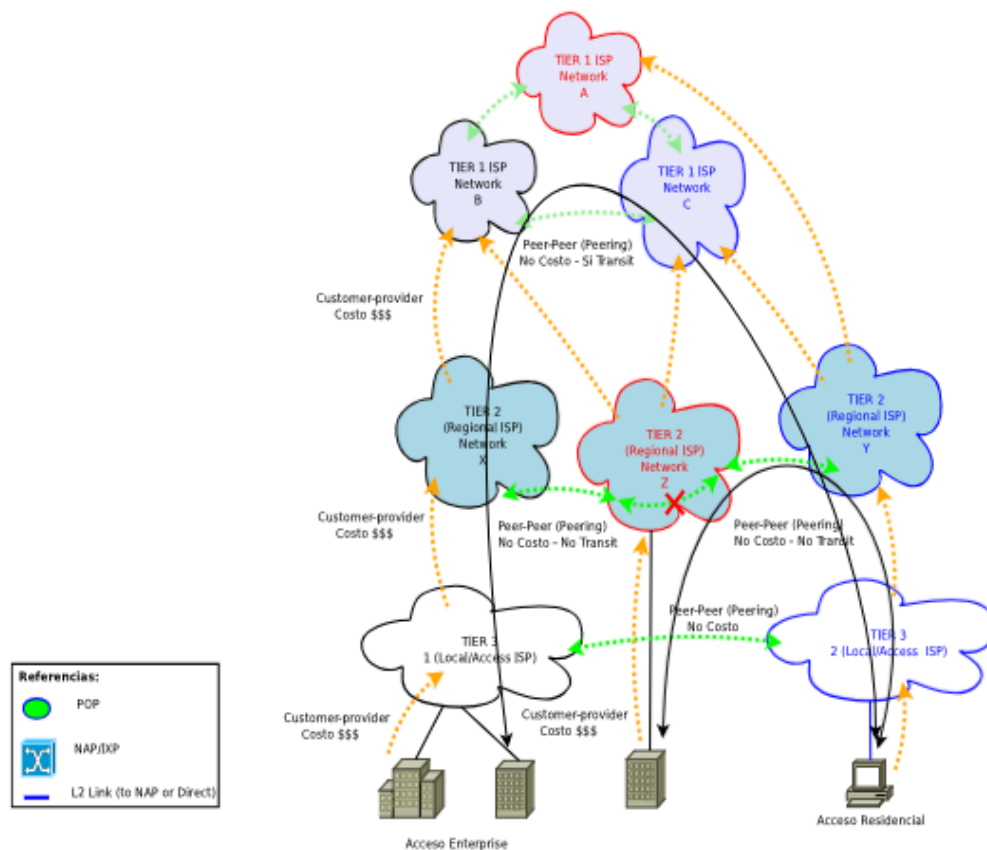
- Categoría STANDARD TRACK. RFC maturity levels.
Proposed Standard: no se requiere implementaciones. Se asigna RFCnnnn.
Internet Standard (STD): existen implementaciones y significativa experiencia operacional. Se retiene el RFCnnnn y se agrega STDxxxx
- Otras Categorías: "Off-track".
INFORMAL/EXPERIMENTAL: otro proceso, se publica como Internet Draft, pero se coloca en otra Cat.
BCP (Best Current Practices): otro proceso. HISTORIC (STD obsoletas): las RFCs se van actualizando o se pueden declarar obsoletas por otras.
FYI: (For Your Information): como INFORMATIONAL

Estructura de Internet

- Estructura en jerarquía, en Tiers.
- Capa de acceso(Edge): Acceso Residenciales, Acceso de Organizaciones.
- Capa de núcleo (Core): dividida en diferentes niveles.
 - Proveedores Regionales
 - Proveedores Nacionales
 - Proveedores Internacionales
 - Proveedores Internacionales en el Tier 1

Core de la estructura

- Tecnologías de fibra óptica, Cobre, Satélites
- Se interconectan mediante POPs (Point Of Presence) con Proveedores.
- Entre proveedores se interconectan mediante NAPs (Network Access Point) o conexiones Directas. Actualmente los NAPs se los llama IXPs (Internet Exchange Point)



Tecnologías de Internet

- Diferentes tecnologías de última milla(acceso) y de redes locales.
- Despliegue local y de acceso de las personas y las organizaciones.
- Ver capa física y enlace modelo OSI.
- Últimos eslabones en la jerarquía

Clase 2

Protocolo HTTP(hasta 1.1) y WEB-Cache

Historia

- WWW (red de sistemas de hipertexto inter-linkados accesibles vía Internet).
- Desarrollada en 1990 por Tim Berners-Lee en el CERN.
 - Desarrolla el protocolo HTTP y el lenguaje HTML.
- 1993: el primer cliente/browser GUI: Mosaic.
- 1994: Netscape Navigator 1.0.
- Desarrollo de servidores, por ejemplo 1995: Apache Server.
- Buscadores, Indexadores son parte importante del servicio.
- Modelo anterior de interacción: servidor produce, cliente consume.
- Nuevos Modelos: Web 2.0, Tim O'Reilly en 2004, modelo de interacción entre usuarios, consumen y producen mediante servicios especiales: blogs, redes sociales, wikis, multimedia, et

Elementos WEB

- Recurso u Objeto HTTP: ej. web page.
- Referenciado por una URI (Uniform Resource Id): URL (Uniform Resource Location) o URN (Name).
- Formato de URL:
 - protocol://[user:pass@]host:[port]/[path].
- Ejemplo: <http://www.NN.unlp.edu.ar:8080/dir/index.html>.
- URN: no indica ubicación, solo identifican, categorías, poco impl. ej. urn:isbn:0132856204, urn:ietf:rfc:2616.
- Objetos pueden ser página web (web page), imágenes JPEG, PNG, GIF, Java Applet, archivos de multimedia: MP3, AVI, etc.

- Páginas web: archivo HTML que incluye v ínculos o directamente otros objetos

HTTP

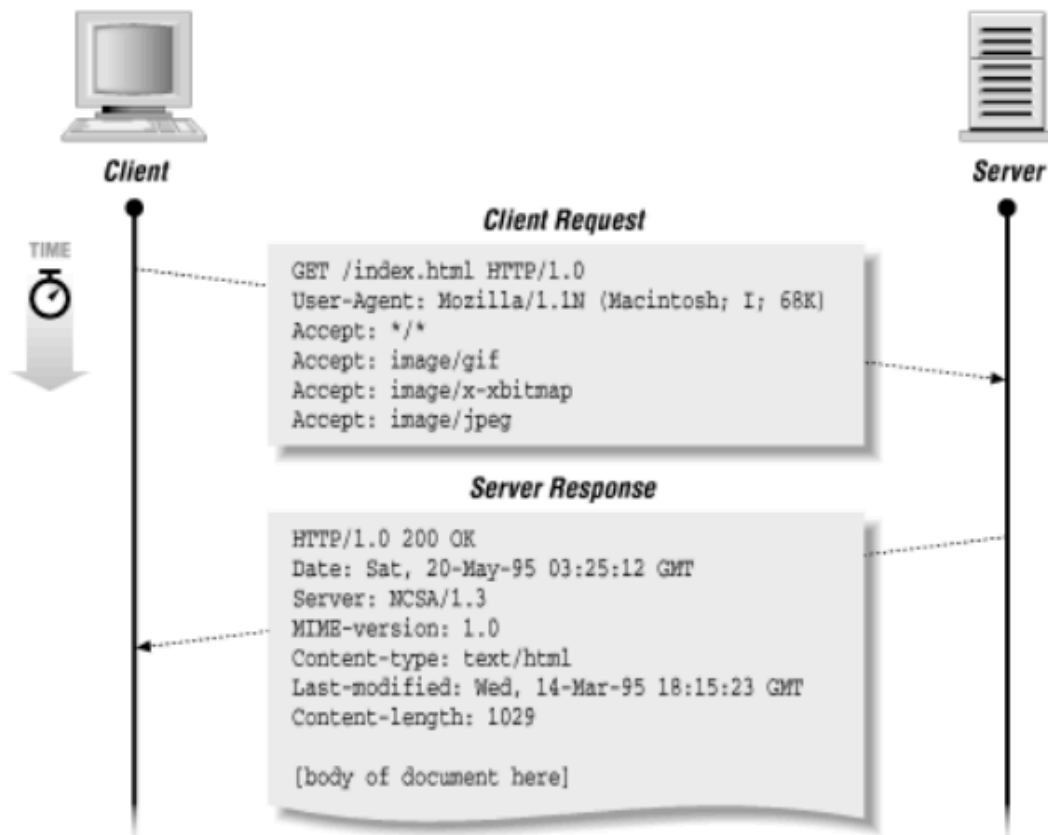
Hypertext Transfer Protocol (HTTP) (o Protocolo de Transferencia de Hipertexto en español) es un protocolo de la [capa de aplicación](#) para la transmisión de documentos hipermedia, como HTML. Fue diseñado para la comunicación entre los navegadores y servidores web, aunque se puede utilizar para otros propósitos también

Funcionamiento HTTP

Funcionamiento HTTP

- Modelo cliente/servidor, Request/Response (sin estados -stateless)
- Protocolo que corre sobre TCP (requiere protocolo de transporte confiable), usa el puerto 80 por default.
- El cliente escoge cualquier puerto no privilegiado.
- Trabaja sobre texto ASCII, permite enviar información binaria con encabezados MIME.
- Clientes (llamados browsers o navegadores): Firefox, IE, Opera, Safari, Chrome
- Servidores: Apache Server, MS IIS, NGINX , Google GWS, Tomcat

Esquema

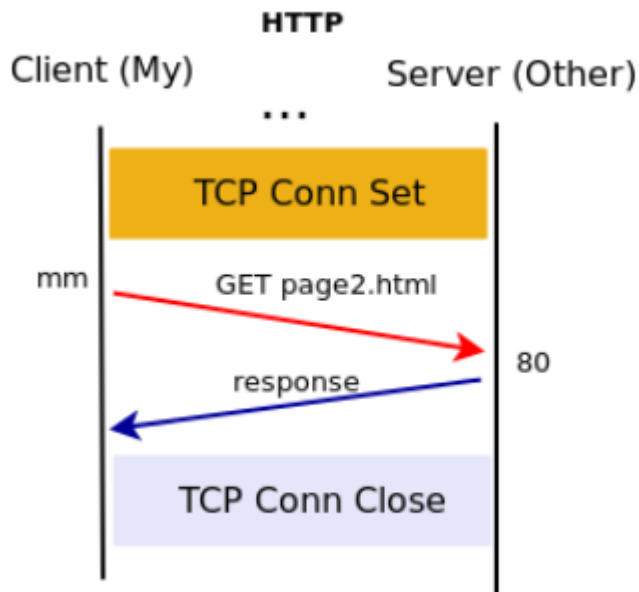


Versión HTTP 0.9

Versión HTTP 0.9

- Primera versión de HTTP fue la 0.9 nunca se estandarizó.
- Pasos para obtener un documento:
 1. Establecer la conexión TCP
 2. HTTP Request vía comando GET.
 3. HTTP Response enviando la página requerida.
 4. Cerrar la conexión TCP por parte del servidor.
 5. Si no existe el documento o hay un error directamente se cierra la conexión

Diagrama de pasos HTTP



- Solo una forma de Requerimiento.
- Solo una forma de Respuesta.
- Request/Response sin estado.

Request ::= GET <document-path> <CR><LF>

Response ::= ASCII chars HTML Document.

GET /hello.html <CR><LF>

GET / <CR><LF>

Versión HTTP 1.0

Versión HTTP 1.0

- Versión de HTTP 1.0 estándar [RFC-1945]
- Define formato, proceso basado en HTTP 0.9:
 - Se debe especificar la versión en el requerimiento del cliente.
 - Para los request, define diferentes métodos HTTP.
 - Define códigos de respuesta.
 - Admite repertorio de caracteres, además del ASCII, como ISO-8859-1, UTF-8, etc.
 - Admite MIME (No solo sirve para descargar HTML e imágenes).
 - Por default NO utiliza conexiones persistentes..

Request

```
<Method> <URI> <Version>
[<Headers Opcionales>]
<Blank>
[<Entity Body Opcional>]
<Blank>
<Method HTTP 1.0> ::= GET, POST, HEAD, PUT,
DELETE, LINK, UNLINK

<HTTP Version> <Status Code> <Reason Phrase>
[<Headers Opcionales>]
<Blank>
[<Entity Body Opcional>]
```

Ejemplos de respuestas

```
HTTP/<version> 200 OK
HTTP/<version> 301 Moved Permanently
HTTP/<version> 400 Bad Request
HTTP/<version> 403 Access Forbidden
HTTP/<version> 404 Not Found
HTTP/<version> 405 Method Not Allowed
HTTP/<version> 500 Internal Server Error (CGI Error)
HTTP/<version> 501 Method Not Implemented
```

Métodos

- **GET**: obtener el documento requerido. Puede enviar información, pero no demasiada. Es enviada en la URL. Formato ? var1=val1&var2=val2.... Limitación de tamaño de URL por parte de las implementaciones. NO espera recibir datos en body.
- **HEAD**: Idéntico a GET, pero sólo requiere la meta información del documento, por ejemplo, su tamaño. Usado por clientes de caché.

- **POST**: hace un requerimiento de un documento, pero también, envía información en el Body. Generalmente, usado en el fill-in de un formulario HTML(FORM). Puede enviar mucha más información que un GET.
- **PUT**: usado para reemplazar un documento en el servidor. En general, deshabilitada. Utilizado, por ejemplo, por protocolos montados sobre HTTP, como WebDAV [WDV].
- **DELETE**: usado para borrar un documento en el servidor. En general, deshabilitada. También, puede ser utilizada por WebDAV.
- **LINK, UNLINK**: establecen/des-establecen relaciones entre documentos.

GET Response

```
GET /index2.html HTTP/1.0
User-Agent: telnet/andres (GNU/Linux)
Host: estehost.com
Accept: */*

HTTP/1.1 200 OK
Date: Mon, 21 Apr 2008 00:28:51 GMT
Server: Apache/2.2.4 (Ubuntu)
Last-Modified: Mon, 21 Apr 2008 00:18:14 GMT
ETag: "a3b36-1f-91d5d80"
Accept-Ranges: bytes
Content-Length: 31
Connection: close
Content-Type: text/plain
```

HEAD Response

```
HEAD /index.html HTTP/1.0
User-Agent: telnet/andres (GNU/Linux)
Host: estehost.com
Accept: */*
HTTP/1.1 200 OK
Date: Mon, 21 Apr 2008 00:40:25 GMT
```

```
Server: Apache/2.2.4 (Ubuntu)
Last-Modified: Mon, 21 Apr 2008 00:18:14 GMT
ETag: "a3b36-1f-91d5d80"
Accept-Ranges: bytes
Content-Length: 31
Connection: close
Content-Type: text/plain
```

```
<HTML>
<H1> HOLA </H1>
...
</HTML>
```

POST Response

```
POST /index.html HTTP/1.0
User-Agent: telnet/andres (GNU/Linux)
Host: estehost.com
Accept: */*
Content-Type: text/plain
Content-Length: 10
1234567890
HTTP/1.1 200 OK
Date: Mon, 21 Apr 2008 00:37:22 GMT
Server: Apache/2.2.4 (Ubuntu)
Last-Modified: Mon, 21 Apr 2008 00:18:14 GMT
ETag: "a3b36-1f-91d5d80"
Accept-Ranges: bytes
```

```
Content-Length: 31
Connection: close
Content-Type: text/plain
```

```
<HTML>
<H1> HOLA </H1>
```

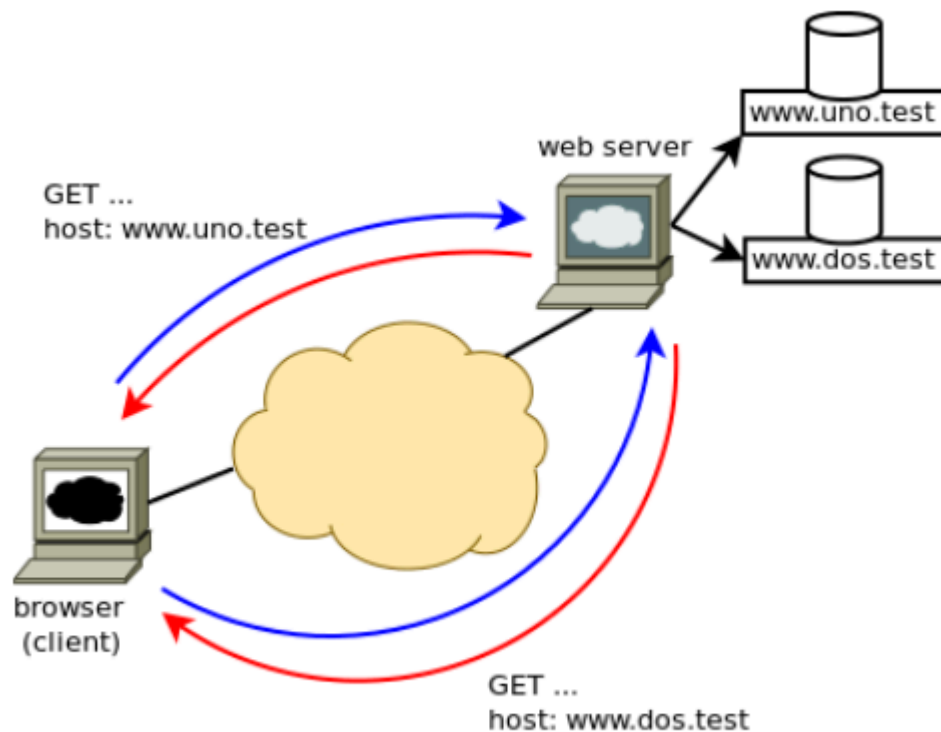
```
...  
</HTML>
```

GET con host virtuales

Mediante el parámetro Host se pueden multiplexar varios servicios sobre un mismo host.

```
? ./mozilla http://www.uno.test  
? ./mozilla http://www.dos.test  
? host www.uno.test  
www.uno.test has address 192.168.0.2  
? host www.dos.test  
www.dos.test has address 192.168.0.2
```

Hosts Virtuales



Autenticación HTTP

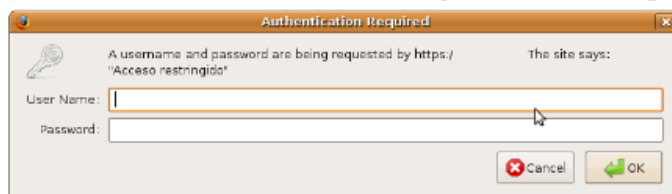
- HTTP/1.0 contempla autenticación con WWW-Authenticate Headers.

- Encabezados, el cliente y el servidor intercambiar información auth.
- El servidor, ante un requerimiento de un documento que requiere autenticación, enviará un mensaje 401 indicando la necesidad de autenticación y un Dominio/Realm.
- El navegador solicitará al usuario los datos de user/password (si es que no los tiene cachedos) y los enviará en texto claro al servidor.
- El servidor dará o no acceso en base a esos valores.
- Para los siguientes requerimientos, el navegador usará los valores que tiene almacenados para el Realm solicitado.

Authorization Required

This server could not verify that you are authorized to access the document requested. Either you supplied the wrong credentials (e.g., bad password), or your browser doesn't understand how to supply the credentials required.

Apache/2.2.9 (Debian) DAV/2 SVN/1.5.1 PHP/5.2.6-1+lenny3 with Suhosin-Patch mod_ssl/2.2.9 OpenSSL/0.9.8g Server at Port 443



Conexiones persistentes

- En HTTP/1.0 no se contemplaron las conexiones persistentes por default.
- A partir de HTTP/1.1 [RFC-2068], si.
- En HTTP/1.0 se pueden solicitar de forma explícita.

```
GET /index.html HTTP/1.0
Connection: Keep-Alive
User-Agent: telnet/andres (GNU/Linux)
Host: estehost.com
Accept: */*
HTTP/1.1 200 OK
Date: Tue, 22 Apr 2008 01:31:56 GMT
Server: Apache/2.2.4 (Ubuntu)
Last-Modified: Mon, 21 Apr 2008 00:18:14 GMT
```

```
ETag: "a3b36-1f-91d5d80"
Accept-Ranges: bytes
```

```
Content-Length: 31
Keep-Alive: timeout=15, max=10
Connection: Keep-Alive
Content-Type: text/html
<HTML>
<H1> HOLA </H1>
</HTML>
GET /index.html HTTP/1.0
Connection: Keep-Alive
...
Connection closed by foreign host.
```

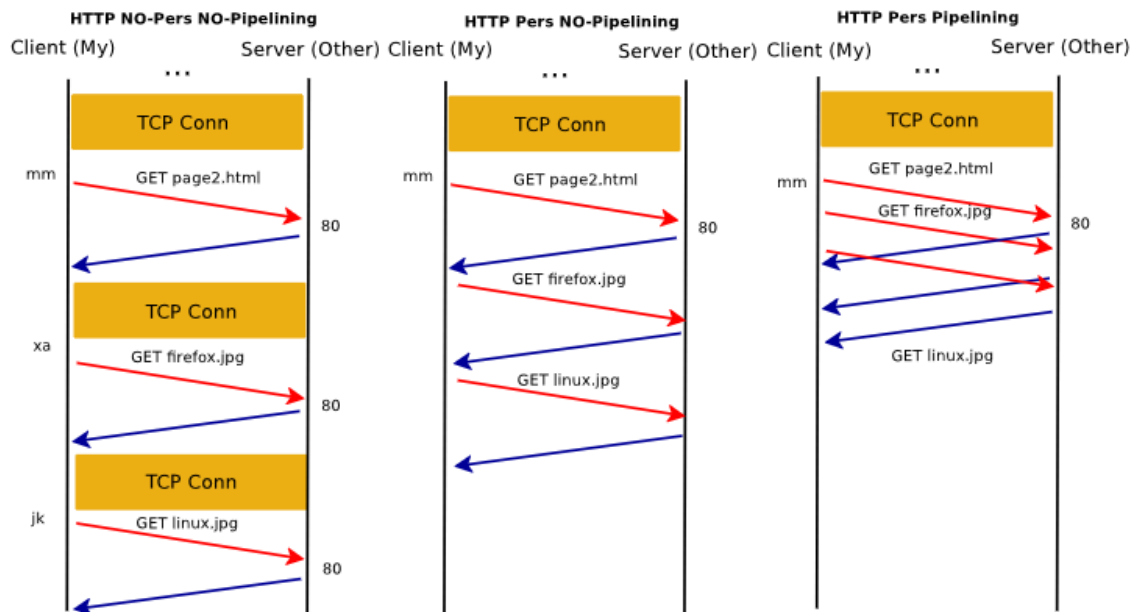
Versión HTTP 1.1

Versión HTTP 1.1

- HTTP/1.1, 1997 con la [RFC-2068] se actualiza con [RFC-2616].
- Nuevos mensajes HTTP 1.1: OPTIONS, TRACE, CONNECT.
- Conexiones persistentes por omisión.
- Pipelining, mejora tiempo de respuestas.

Pipelining

- No necesita esperar la respuesta para pedir otro objeto HTTP.
- Solo se utiliza con conexiones persistentes.
- Mejora los tiempos de respuestas.
- Sobre la misma conexión se debe mantener el orden de los objetos que se devuelven.
- Se pueden utilizar varios threads para cada conexión.
 - Sin pipelining: $1RTT + FT$ por cada objeto, n objetos $nRTT + nFT$.
 - Con pipelining óptimo: n objetos: $1RTT + nFT$.



Trace y connect HHTTP 1.1

- TRACE utilizada para debugging.
- El servidor debe copiar el requerimiento tal cual.
- El cliente puede comparar lo que envía con lo que recibe.
- CONNECT utilizada para generar conexiones a otros servicios montadas sobre HTTP.
- Proxy-Agent genérico

Ejemplo CONNECT

```
CONNECT 127.0.0.1:25
HTTP/1.0 200 Connection Established
Proxy-agent: Apache/2.2.4 (Ubuntu)
220 khartum ESMTTP Postfix (Ubuntu)
HELO otrohost.com
250 khartum
QUIT
221 2.0.0 Bye
```

Redirects HTTP

- Redirect temporal 302, indicando la nueva URL/URI

- El user-agent no debería re-direccionarlo salvo que el usuario confirme
- Moved Permanently 301, se indica que cualquier acceso futuro debe realizarse sobre la nueva ubicación (mejora indexadores)
- Se pueden generar problemas con Cookies

Otras Características de HTTP

CGIs Scripts y JavaScript

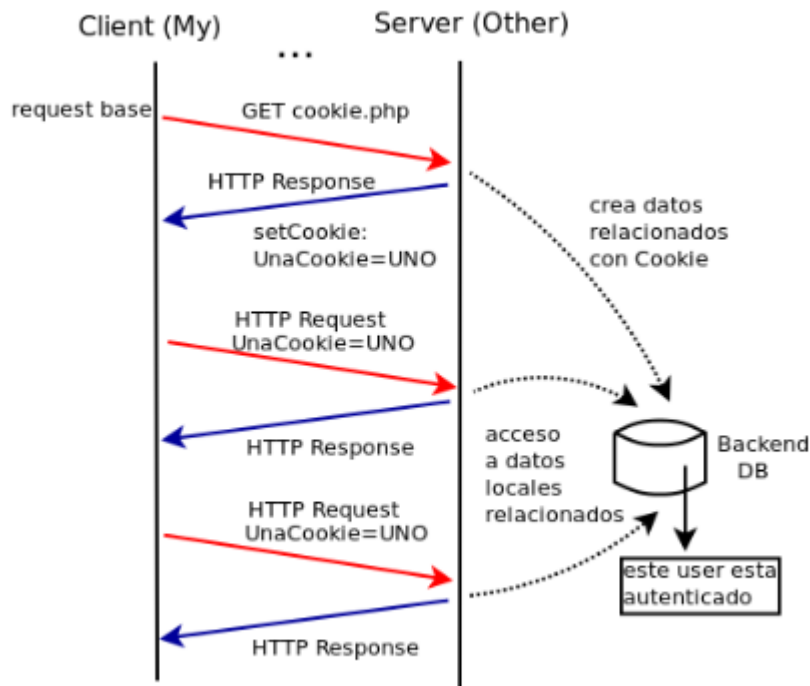
- Necesidad de dinamismo para generar aplicaciones.
- Server-Side Script:
 - Ejecuta del lado del servidor.
 - CGI (Common Gateway Interface): aplicación que interactúa con un servidor web.
 - CGIs leen de STDIN (POST) o de variables de entorno (GET): QUERY STRING datos de usuario.
 - Escriben en la STDOUT response.
 - Deben anteponer el content-type en el header.
 - POST permite enviar más datos.
 - Lenguajes de scripting más flexibles y seguros: PHP, ASP, JSP.
 - Implementados como CGIs, dentro o módulos propios del web-server.
- Client-Side Script
 - Ejecuta del lado del cliente, en el browser
 - JavaScript estándar W3C
 - Usan modelo de objetos DOM (Document Object Model)
 - Otros lenguajes JScript, VBScript.
 - Permiten extensiones como AJAX(Asynchronous JavaScript And XML)
 - AJAX hace requerimientos particulares y no necesita recargar toda la página.
 - Parseo XML para comunicarse.
 - Existen numerosos frameworks que encapsulan esta funcionalidad brindando una interfaz de programación, API fácil

de utilizar.

- Server-Side to Server-Side Script:
 - Permiten comunicación entre servidores.
 - Modelo de “objetos” y servicios distribuidos.
 - Conjunto de convenciones para implementar RMI (Remote Method Invocation) sobre HTTP (u otro protocolo de texto).
 - Previo XML-RPC.
 - SOAP (Simple Object Access Protocol).
 - Web-Services.
 - REST.

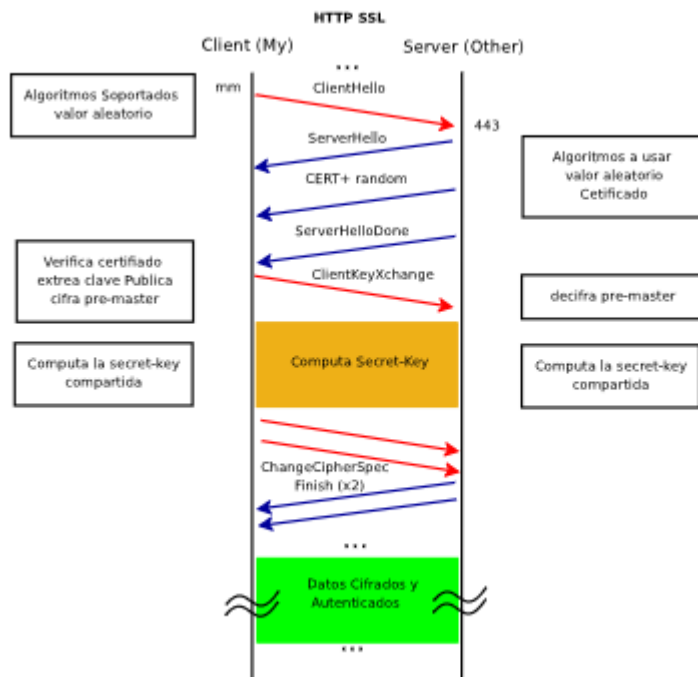
Cookies

- Mecanismo que permite a las aplicaciones web del servidor “manejar estados”.
- El cliente hace un request.
- El servidor retorna un recurso (un objeto HTTP, como una página HTML) indicando al cliente que almacene determinados valores por un tiempo.
- La Cookie es introducida al cliente mediante el mensaje en el header Set-Cookie: mensaje que indica un par (nombre,valor).
- El cliente en cada requerimiento luego de haber almacenado la Cookie se la enviará al servidor con el header Cookie:.
- El servidor puede utilizarlo o no.
- El servidor puede borrarlo.
- Esta información puede ser usada por client-side scripts.



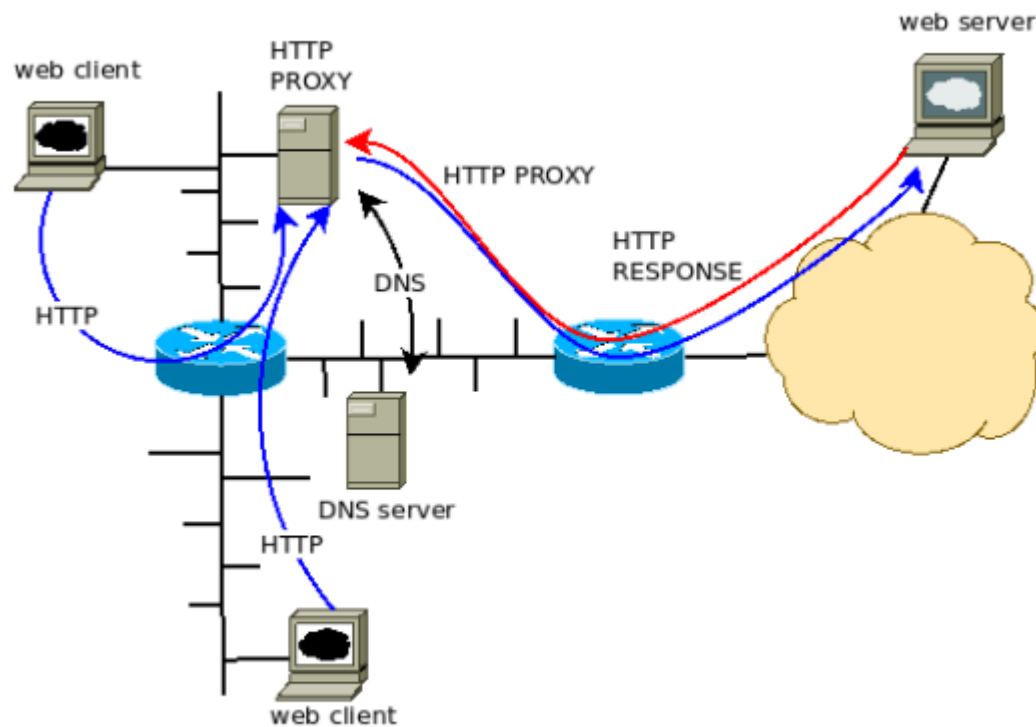
HTTPS (HTTP sobre TLS/SSL)

- Utiliza el port 443 por default.
- Etapa de negociaci3n previa.
- Luego se cifra y autentica todo el mensaje HTTP (incluso el header)



WEB-Cache

- "Proxiar" y Cachear recursos HTTP
- Objetivos:
 - Mejorar tiempo de respuesta (reducir retardo en descarga).
 - Ahorro en BW (recursos de la red)
 - Balance de carga, atender a todos los clientes
- Se solicita el objeto, si esta en cache y está "fresco" se retorna desde allí (HIT)
- Si el objeto no está o es viejo se solicita al destino y se cachea
- Se puede realizar control de acceso
- Cache del lado del cliente (privada).
- Los web browser tienen sus propias cache locales.
- Los servidores agregan headers:
 - Last-Modified: date
 - ETag: (entity tag) hash
- Requerimientos condicionales desde los clientes:
 - If-Modified-Since: date
 - If-None-Match: hash
- Respuestas de los servidores:
 - 304 Not Modified.
 - 200 OK
- Los cache como servers funcionan como Proxy (shared cache).
- Son servidores a los clientes y clientes a los servidores web.
- Los instalan ISP o redes grandes que desean optimizar el uso de los recursos.
- Existen:
 - Proxy no-transparente.
 - Proxy transparentes.
 - Proxy en jerarquía o mesh (ICP y HTCP).
 - CDN (Content Delivery Network), funcionan por DNS.



HTTP/2 y HTTP/3

HTTP-2

¿Qué es?

- Reemplazo de cómo HTTP se transporta.
- No es un reemplazo del protocolo completo.
- Se conservan métodos y semántica.
- Base del trabajo protocolo desarrollado por Google SPDY/2.
- Definido en:
 - RFC7540: Hypertext Transfer Protocol version 2.
 - RFC7541: HPACK - Header Compression for HTTP/2

Problemas con HTTP/1.0, HTTP/1.1

- Un request por conexión, por vez, muy lento.
- Alternativas (evitar HOL/HOLB Head of Line Blocking):
 - Conexiones persistentes y pipelining.
 - Generar conexiones paralelas.

- Problemas:
 - Pipelining requiere que los responses sean enviado en el orden solicitado, HOLB posible.
 - POST no siempre pueden ser enviados en pipelining.
 - Demasiadas conexiones genera problemas, control de congestión, mal uso de la red.
 - Muchos requests, muchos datos duplicados (headers).

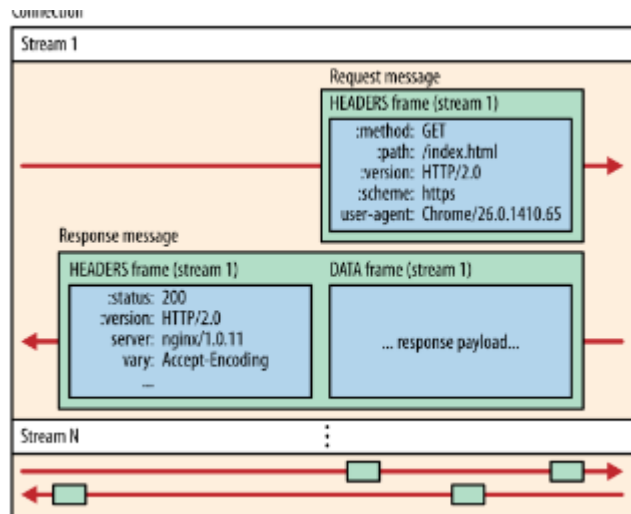
Diferencias principales con HTTP/1.1

- Protocolo binario en lugar de textual (ASCII), binary framing (más eficiente).
- Multiplexa varios request en una petición en lugar de ser una secuencia ordenada y bloqueante
- Utilizar una conexión para pedir/traer datos en paralelos, agrega: datos fuera de orden, priorización, flow control por frame

HTTP/2 mux stream framing

- Puede generar una o más conexiones TCP. Trata de aprovechar las que tiene establecidas.
- Un stream es como una sub-conexión (una "conexión" http2 dentro de una conexión TCP).
- Un stream tiene un ID y una prioridad(alternativa) y son bidireccionales.
- Sobre una conexión TCP multiplexa uno o más streams ("conexiones http2").
- Los streams transportan mensajes.
- Los mensajes http2 (Request, Response) se envían usando un stream,
- Los mensajes http2 son divididos en frames dentro del mismo stream.
- Un frame es una porción de mensaje: header fijo + payload variable (unidad mínima).
- El mismo stream puede ser usado para llevar diferentes msj.

- Los mensajes están compuestos por frames, que podrían ser de diferentes tipos.
- Los streams van en una misma conexión.
- Los streams son identificados y divididos en frames.
- Frame types: HEADERS, DATA, PUSH_PROMISE, WINDOW_UPDATE, SETTINGS, etc.



http2_native.pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http2

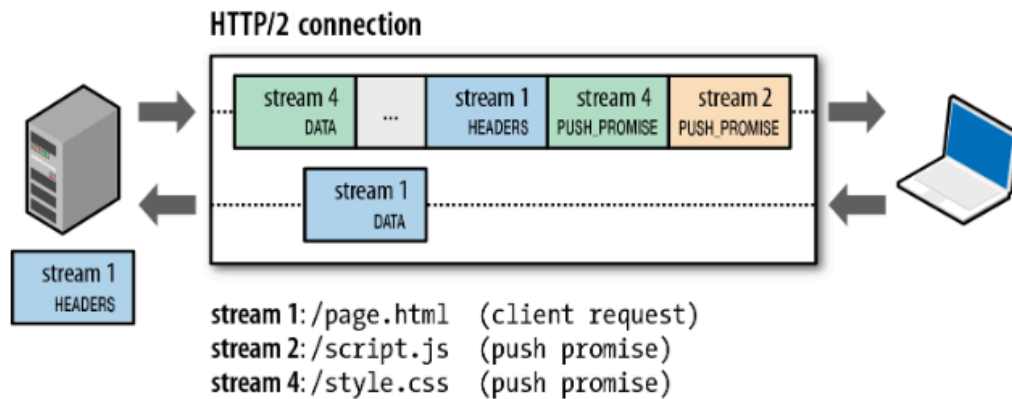
No.	Time	Source	Destination	Protocol	Length	Info
4	0.414129282	192.168.0.228	139.162.123.134	HTTP2	90	Magic
5	0.420556333	192.168.0.228	139.162.123.134	HTTP2	152	SETTINGS[0], WINDOW_UPDATE[0], HEADERS[1]: G
7	0.839101976	139.162.123.134	192.168.0.228	HTTP2	100	SETTINGS[0]
10	0.839139220	139.162.123.134	192.168.0.228	HTTP2	76	SETTINGS[0]
12	0.839167233	139.162.123.134	192.168.0.228	HTTP2	369	HEADERS[1]: 200 OK, DATA[1] (text/plain)
14	0.839200935	192.168.0.228	139.162.123.134	HTTP2	75	SETTINGS[0]
15	0.845040952	192.168.0.228	139.162.123.134	HTTP2	93	HEADERS[3]: GET /humans.txt
16	0.845040952	192.168.0.228	139.162.123.134	HTTP2	134	HEADERS[3]: 404 Not Found, DATA[3] (text/plain)

Transmission Control Protocol, Src Port: 38564, Dst Port: 80, Seq: 25, Ack: 1, Len: 86

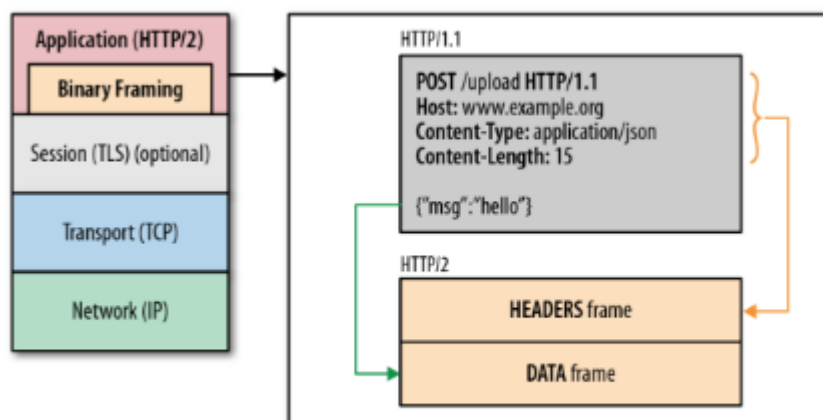
HyperText Transfer Protocol 2

- Stream: SETTINGS, Stream ID: 0, Length 18
- Stream: WINDOW_UPDATE, Stream ID: 0, Length 4
- Stream: HEADERS, Stream ID: 1, Length 37, GET /robots.txt
 - Length: 37
 - Type: HEADERS (1)
 - Flags: 0x05
 - 0... .. = Reserved: 0x0
 - .000 0000 0000 0000 0000 0000 0001 = Stream Identifier: 1
 - [Pad Length: 0]
 - Header Block Fragment: 82048862c3c674a174f94f864188aa69d29ac4b9ec9b7a88...
 - [Header Length: 136]
 - [Header Count: 6]
 - Header: :method: GET
 - Header: :path: /robots.txt
 - Header: :scheme: http
 - Header: :authority: nghttp2.org
 - Header: user-agent: curl/7.61.0
 - Header: accept: */*

HyperText Transfer Protocol 2: Protocol Packets: 21 - Displayed: 8 (38.1%) Profile: Classic



- Streams codificados en binario y cada frame con header común fijo(9B).

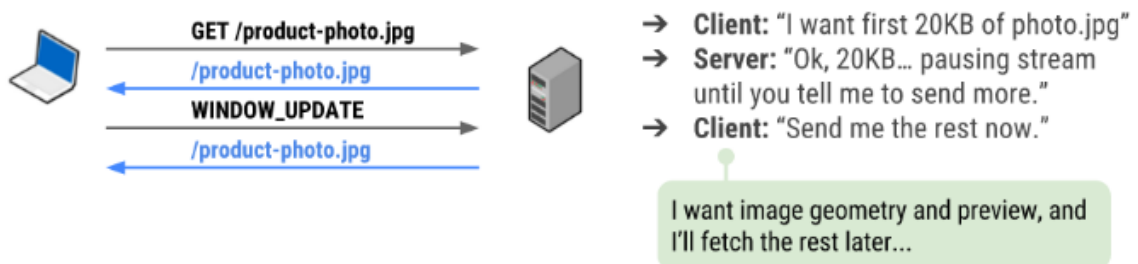


Headers

- Se mantienen casi todos los HEADERS de HTTP/1.1.
 - No se codifican más en ASCII.
 - Surgen nuevos pseudo-headers que contiene información que estaba en el método y otros headers.
 - Por ejemplo: HEAD /algo HTTP/1.1 se reemplaza con http2:
 - :method: head
 - :path: /algo
 - :scheme: https o http
 - :authority: www.site.com reemplaza al headerHost:.
- Para las respuestas: :status: códigos de retornos 200, 301, 404, etc.

Priorización y flow-control

- Los streams dentro de una misma conexión tienen flow-control individual.
- Los streams pueden tener un weight (prioridad).
- Los streams pueden estar asociados de forma jerárquica, dependencias.



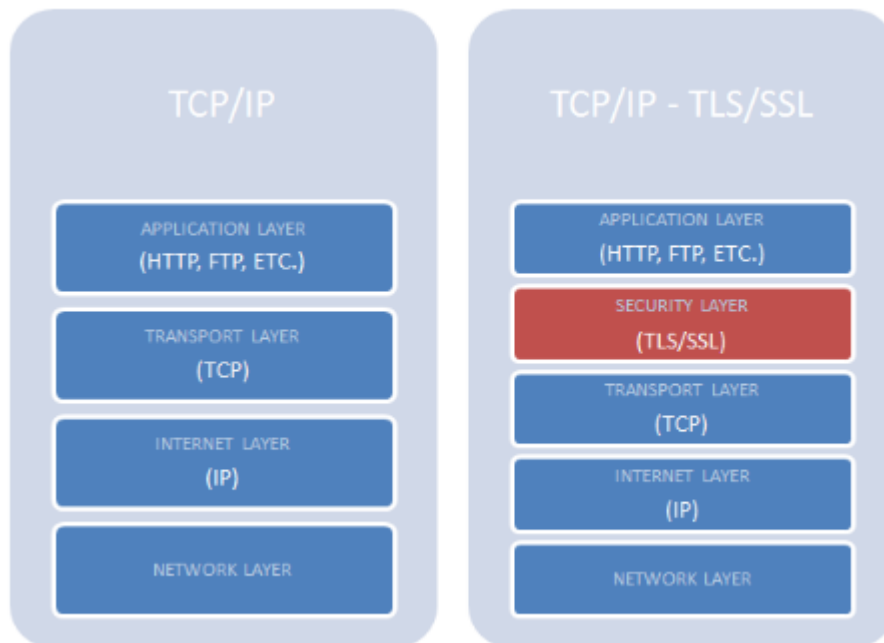
Comprensión y soporte

- Comprensión de encabezados
- SPDY/2 propone usar GZIP.
- GZIP + cifrado, tiene "bugs" utilizados por atacantes.
- Se crea un nuevo compresor de Headers: HPACK.
- H2 y SPDY, soportados en la mayoría de los navegadores.

Otras características

- HTTP/1.1, posibilidad de hacer un upgrade durante la conexión:
 - Upgrade Header. Connection: Upgrade, HTTP2-Settings
Upgrade: h2c|h2.
- http2: Negociar el protocolo de aplicación: ALPN: Application-Layer Protocol Negotiation.
Se negocia como extensión de SSL en Hello (Anteriormente NPN).
Se ofrece: h2, h3, http/1.1, ...
- Posibilidad de negociar protocolo alternativo:
Alterantive Service: alt-svc.

SSL/TLS



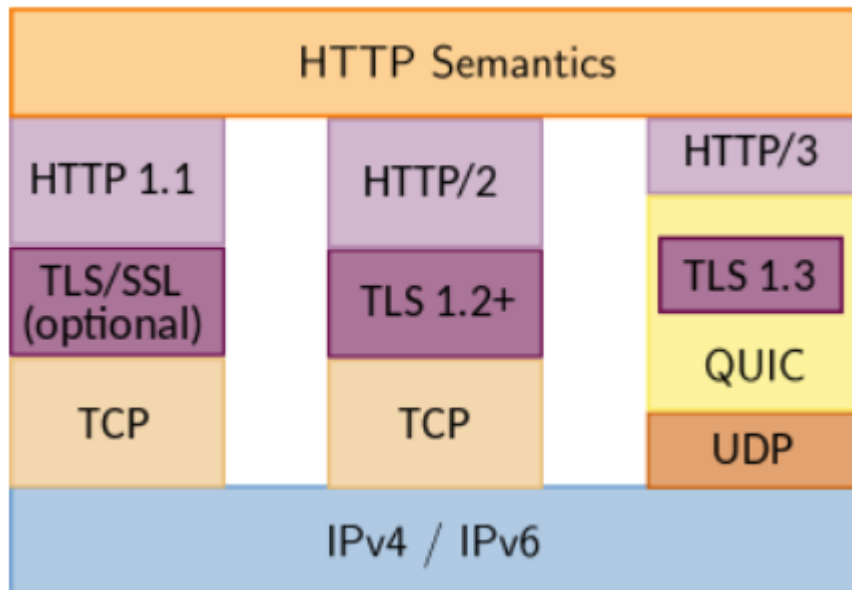
Capa SSL/TLS (Seguridad)

- **Qué es:** TLS es el estándar actual (SSL ya está obsoleto).
- **Función principal:** Actúa como intermediario. Encripta la información que sale de la capa de Aplicación antes de dársela a TCP, y la desencripta cuando entra.
- **En la práctica:** La pila izquierda representa **HTTP** (texto plano) y la derecha **HTTPS** (tráfico seguro).

HTTP-3

¿Qué es?

HTTP/3 es la última evolución del protocolo web, cuya principal novedad es el reemplazo de TCP por **QUIC**, un protocolo de transporte seguro que opera sobre **UDP** (puerto 443 por defecto). Aunque cambia su infraestructura base para mejorar el rendimiento y la latencia, conserva intactos los métodos y la semántica de HTTP/1.1, al igual que su predecesor HTTP/2. Está estandarizado en los **RFC 9114** (HTTP/3) y **RFC 9000** (QUIC)



Diferencias principales de HTTP/3-QUIC

- Reducción de latencia en conexiones (2 msj. de handshake)
- No depende del manejo de la conexión y controles de TCP (evita HOLB)
- Loss recovery/congestion control, sobre UDP, lo puede hacer como TCP-CUBIC o de otra forma.
- QUIC genere nuevos números de secuencia y re-cifra las retransmisiones, permitiendo mejor detección de pérdidas y medición de RTT
- QUIC paquetes cifrados de forma individual, no por conexión/bytestream.
- QUIC facilita movilidad, tiene ID de conexión independiente de IPs y ports.
- Solo trabaja seguro con cifrado.
- Desventaja de QUIC: muchos middle-boxes filtran UDP, salvo port 53 y NAT. Protocolo de transporte QUIC: implementado usualmente en
- User-space vs Kernel-space.
- UDP facilita algunos ataques de seguridad.

Clase 3

DNS

Principios

Principios e introducción DNS

Aspectos

- DNS cubre los siguientes aspectos:
 - Especifica la sintaxis de los nombres y las reglas para delegar autoridad sobre los nombres.
 - Especifica sistema distribuido para "mapear" nombres con direcciones y otras operaciones.
 - Define la implementación de un protocolo para comunicación de las componentes del sistema.
- Descentralizar el mecanismo de asignación de nombres, aunque sigue sistema jerárquico.
- Delegar autoridad y responsabilidad de asignación y mapeo en organismos intermedios
- Podemos verlo como una base de datos (DB) distribuída.

Elementos

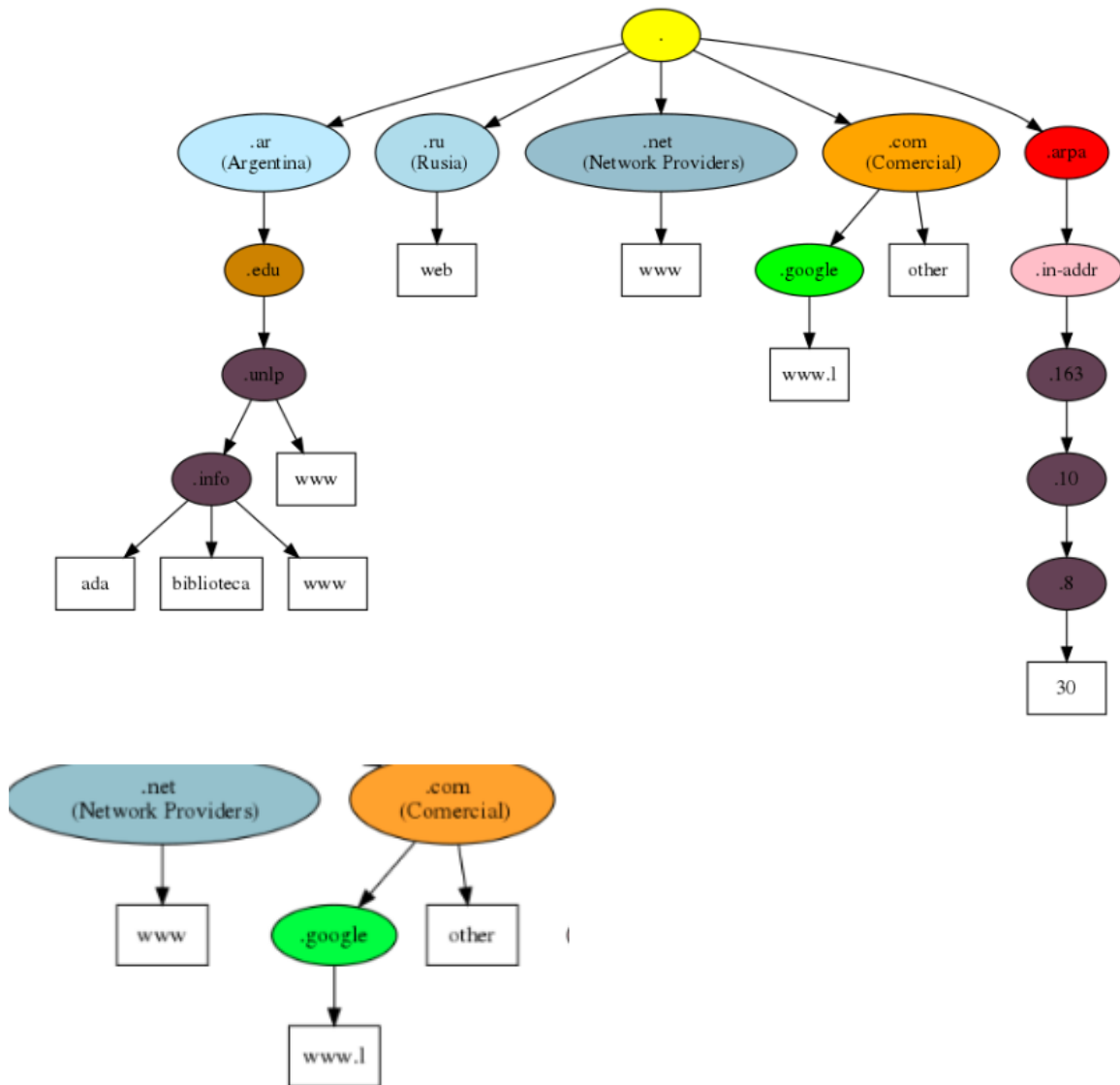
- Nombre de dominio FQDN: lista de etiquetas (labels) separadas por puntos.
- Se leen desde el nodo/etiqueta de la izquierda hasta la raíz del árbol (el punto), estructura jerárquica con sub-nombres (niveles).
- La sintaxis jerárquica refleja la delegación de autoridad.
- No son case-sensitive, cada etiqueta M´aximo 63 chars.
- M´aximo etiquetas 127, nombre no m´as de 255 chars, acepta valores internacionales, UTF-8, Unicode

paraguil (No FQDN)

paraguil.cities.org. (FQDN)

paraguil.cities.org (Considerado FQDN)

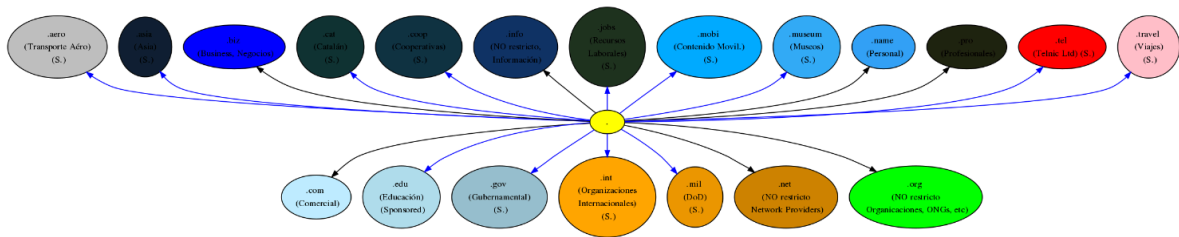
Esquema de nombres



TLDs (Top Level Domains)

- Los TLDs se podrían clasificar en 3 grupos:
 - gTLDs, Generic TLDs: contienen dominios con propósitos particulares, de acuerdo a diferentes actividades.
Políticas definidas por el ICANN: **UNsponsored TLD** o definidas por otra organización: **Sponsored TLD**
 - ccTLD Country-Code TLDs: Contienen dominios delegados a diferentes países del mundo. ISO 3166-1 alfa-2.
 - .ARPA TLD: Es un dominio especial, usado internamente para resolución de reversos.

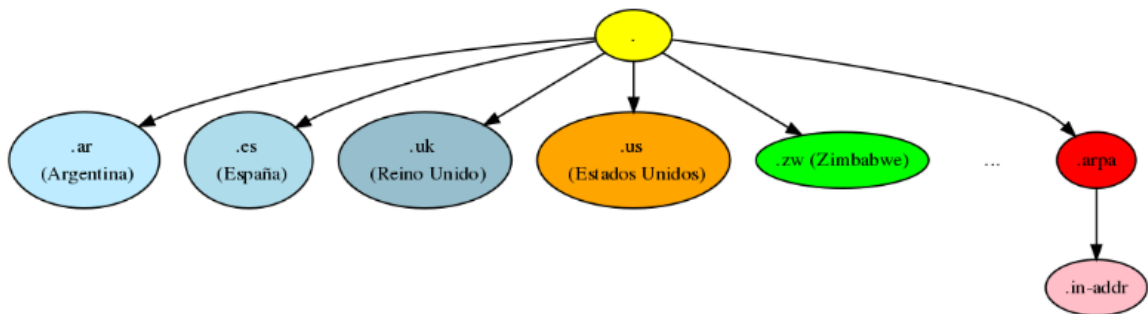
Generic TLDs antes de new gTLDs



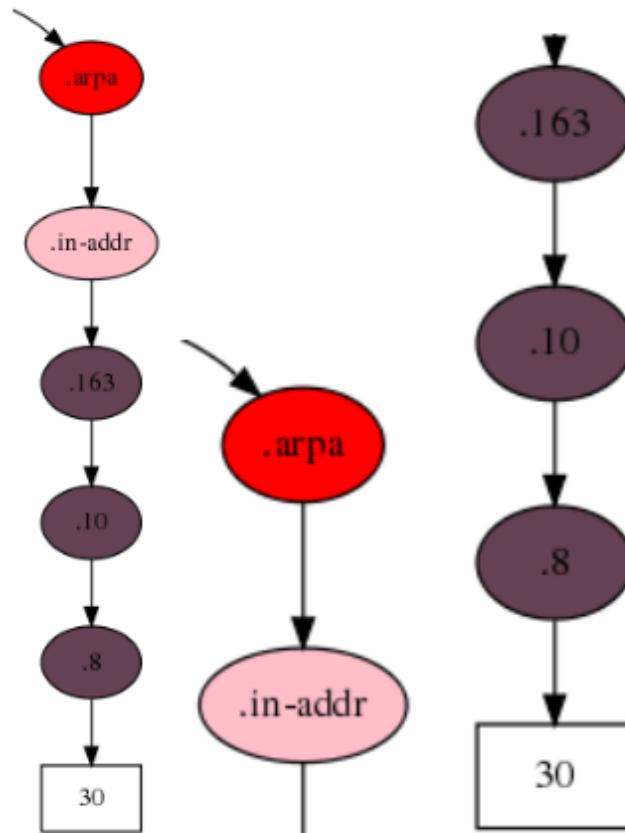
Generic TLDs actualmente

- A partir de 2012 se comenzó a aceptar nuevas aplicaciones para new gTLDs.
- Proceso de licitación donde hay en juego grandes sumas de dinero.
- Nuevos dominios registrados (500+):
 - .academy, .casa, ...
 - .beer, .bike, .futbol,...
 - ...

Country Code y ARPA TLD



Esquema nombres de DNS (ARPA)



```
$ host 163.10.8.30 30.8.10.163.in-addr.arpa domain name
pointer
host163-10-8-30.presi.unlp.edu.ar.
$ host 2800:340:0:64::145
5.4.1.0.0.0.0.0.0.0.0.0.0.0.0.0.4.6.0.0.0.0.0.0.4.3.0.0
.0.8.2.ip6.arpa
domain name pointer www.unlp.edu.ar.
```

Organización

Organización del DNS

- Sistema distribuido y jerárquico.
- Organización mediante dominios, sub-dominios y host o servicios.
- IANA a través del ICANN (Internet Corporation for Assigned Names and Numbers) controla el funcionamiento.
- Existen organizaciones paralelas: Open Root Server Network (ORSN), OpenNIC.
- Delegación mediante RIRs (Regional Internet Registers):
 - American Registry for Internet Numbers (ARIN).

- RIPE NCC -Europa y parte de Asia- (RIPE).
 - Asia-Pacific Network Information Centre (APNIC).
 - Latin American and Caribbean NIC (LACNIC).
 - African Network Information Centre (AfriNIC).
- Nombres se delegan a países, direcciones IP no.



Ejemplo delegación de autoridad

- ada.info.unlp.edu.ar
- “Ada” fue registrada por la administración de la red de la Facultad de Informática.
- El administrador de la Facultad obtuvo previamente la autoridad sobre el dominio “info.unlp.edu.ar”. A partir de la administración de la universidad UNLP.
- La Universidad obtuvo autoridad sobre el dominio “unlp.edu.ar” a partir de la administración de “edu.ar”, RIU (Red Inter-universitaria)
- La RIU obtuvo autoridad sobre “edu.ar” a partir de la delegación de la Cancillería o el ente a cargo de “.AR” (Argentina).
- La administración de nombres en la Argentina, sea la Secretaría Legal y Técnica u otro ente obtuvo la autoridad delegada a partir del IANA o ICANN.

Delegación de dominios

- Actualidad: 13 ROOT Servers distribuidos en todo el mundo.
- Los cuales trabajan con redundancia y las réplicas están distribuidos geográficamente.
- Redundancia, combinada con Ruteo Anycast.

```
. 518400 IN NS A.ROOT-SERVERS.NET. # Versign-grs.com
. 518400 IN NS B.ROOT-SERVERS.NET. # ISI.edu
. 518400 IN NS C.ROOT-SERVERS.NET. # Cogent.com (ANYCAST)
. 518400 IN NS D.ROOT-SERVERS.NET. # UMD.edu (Univ. Maryland)
. 518400 IN NS E.ROOT-SERVERS.NET. # NASA.gov
. 518400 IN NS F.ROOT-SERVERS.NET. # ISC.org (ANYCAST)
. 518400 IN NS G.ROOT-SERVERS.NET. # NIC.mil
. 518400 IN NS H.ROOT-SERVERS.NET. # ARMY.mil
. 518400 IN NS I.ROOT-SERVERS.NET. # NIC.ddn.mil (ANYCAST)
. 518400 IN NS J.ROOT-SERVERS.NET. # Versign-grs.com (ANYCAST)
. 518400 IN NS K.ROOT-SERVERS.NET. # RIPE.net (ANYCAST)
. 518400 IN NS L.ROOT-SERVERS.NET. # ICANN.org (ANYCAST)
. 518400 IN NS M.ROOT-SERVERS.NET. # WIDE.ad.jp (ANYCAST)
```

Funcionamiento DNS

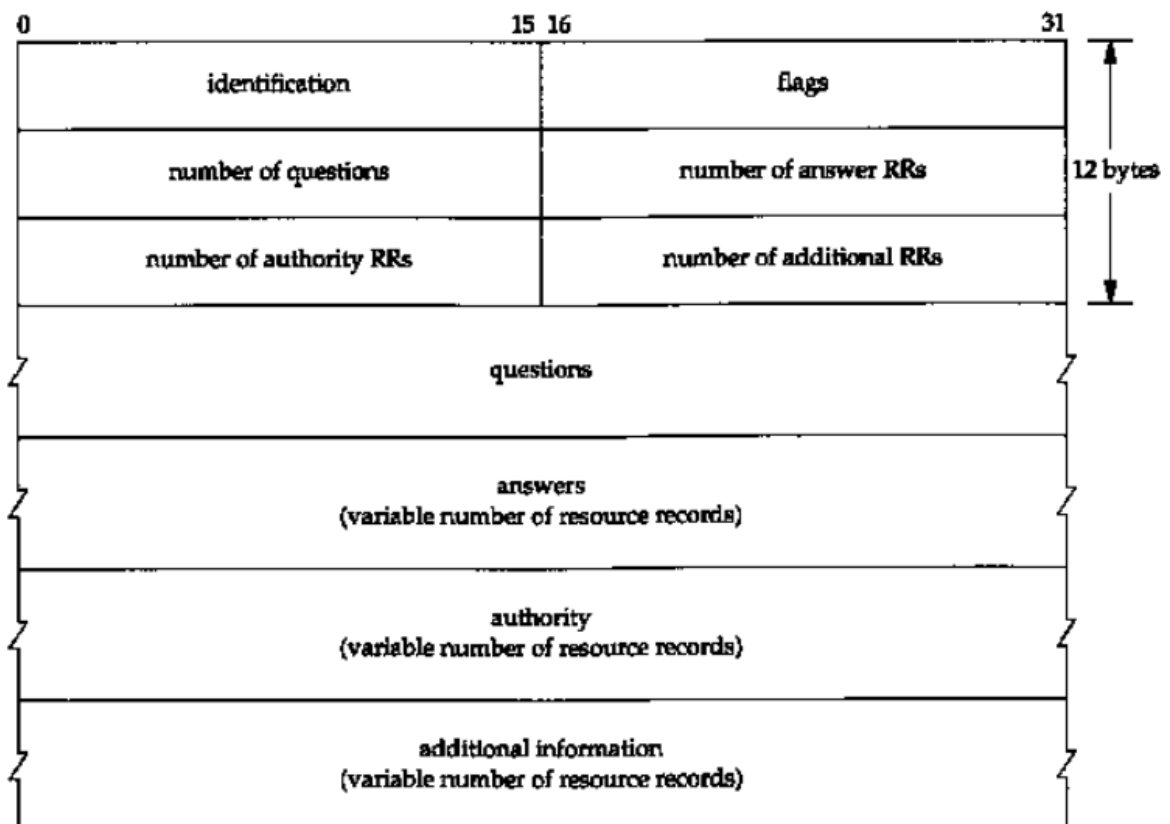
Funcionamiento DNS

Tipos de servidores

- **Servidor Raíz:** Servidor que delega a todos TLD (Top Level Domains). No debería permitir recursivas.
 - **Servidor Autoritativo:** servidor con una zona o sub-dominio de nombres a cargo. podría sub-delegar.
 - **Servidor Local/Resolver Recursivo:** es un servidor que es consultado dentro de una red. mantiene cache. Puede ser Servidor Autoritativo. Permite recursivas “internas”. También llamado Caching Name Server.
 - **Open Name Servers:** servidores de DNS que funcionan como locales para cualquier cliente. Por ejemplo 8.8.8.8, 8.8.4.4, 4.2.2.2, 4.2.2.3
 - Forwarder Name Server: interactúan directamente con el sistema de DNS exterior. Son DNS proxies de otros DNS internos.
 - Servidor Primario y Secundario: solo una cuestión de implementación. donde se modifican los datos realmente
- Modelo cliente/servidor, Request/Response.
 - También hay diálogo entre los servidores.

- Protocolo que corre sobre UDP y TCP, puerto 53
- El cliente escoge cualquier puerto no privilegiado.
- No trabaja sobre texto ASCII
- Si el mensaje supera los 512 bytes se utiliza TCP, e.g. zone transfer.
- Clientes: resolver + cualquier aplicación que requiera la resolución de nombres.
 - Unix el resolver conjunto de funciones C library (libc).
 - Otras implementaciones Smart Resolver servidor Local en cada equipo, caching.
- Servidores: BIND (Berkeley Internet Name Domain/Daemon) de ISC; UNBOUND

Estructura de mensaje de DNS



Resolución de Nombres, Iterativo vs. Recursivo

Registros NS (Name Server): nombre-dom → dns server(s).

Registros SOA (Start Of Authority): params. de dominio.

Registros A (Adress)

```
# less /etc/bind/db.cities.org
...
berlin.cities.org. IN A 172.20.1.100
brasilia.cities.org. IN A 172.20.1.5
paraguil-br0.cities.org. IN A 172.20.1.1
...
```

Registros AAAA (IPv6 Address)

```
# less /etc/bind/db.cities.org
...
berlin.cities.org. IN AAAA 2001:db8:1234:4567::100
brasilia.cities.org. IN AAAA 2001:db8:1234:4567::5
...
```

Registros PTR (Pointer)

```
# less /etc/bind/db.172
...
1.1.20 IN PTR paraguil-br0.cities.org.
5.1.20 IN PTR brasiliass.cities.org.
100.1.20 IN PTR berlin.cities.org.
...
5.1.19 IN PTR sucre.lat.org.
1.1.19 IN PTR paraguil-tap2.lat.org.
...
```

Registros CNAME (Canonical Name)

```
# less /etc/bind/db.cities.org
...
```

```
ftp.cities.org. IN CNAME berlin.cities.org.  
www.cities.org. IN CNAME berlin.cities.org.  
...
```

Registros MX (Mail Exchanger)

```
# less /etc/bind/db.cities.org  
...  
cities.org. IN MX 1 brasilia.cities.org.  
cities.org. IN MX 10 berlin.cities.org.  
...  
# dig -t mx gmail.com  
...  
gmail.com IN MX 5 gmail-smtp-in.l.google.com.  
gmail.com IN MX 10 alt1.gmail-smtp-in.l.google.com.  
gmail.com IN MX 10 alt2.gmail-smtp-in.l.google.com.  
gmail.com IN MX 50 gsmtpl47.google.com.  
gmail.com IN MX 50 gsmtpl83.google.com.  
...
```

Registros NS (Name Server)

```
# less /etc/bind/db.cities.org  
...  
; ## ZONA RAIZ  
cities.org. IN NS berlin.cities.org.  
cities.org. IN NS brasilia.cities.org.  
; ## ZONA delegada  
trees.cities.org. IN NS brasilia.cities.org.  
trees.cities.org. IN NS berlin.cities.org.  
trees.cities.org. IN NS oak.trees.cities.org.  
; ## GLUE RECORD ##  
oak.trees.cities.org. IN A 192.168.40.1  
...
```

Registros SOA (Start Of Authority)

```
$TTL 604800 ; ### TTL global para todos
cities.org. IN SOA berlin.cities.org.
root.berlin.cities.org. (
2008092901 ; ## Serial
604800 ; ## Refresh
86400 ; ## Retry
2419200 ; ## Expiry
604800 ) ; ## Neg Cache TTL
...
```

Ejemplos con DNS

- Consultas con DNS

```
? host -t a berlin.cities.org 127.0.0.1
? dig +nocomments -t a brasilia.cities.org @127.0.0.1
? dig +recurse +short www.unlp.edu.ar @192.112.36.4
? dig +recurse +short www.unlp.edu.ar @127.0.0.1
? dig +short -t ptr 100.1.20.172.in-addr.arpa @127.0.0.1
? host -t mx cities.org 127.0.0.1
? host -t ns cities.org 127.0.0.1
```

TTL y Registros TXT

```
? dig www.unlp.edu.ar | grep -A1 "ANSWER SECTION"
;; ANSWER SECTION:
www.unlp.edu.ar. 155 IN A 163.10.0.145
? dig -t mx gmail.com | grep -A1 "ANSWER SECTION"
;; ANSWER SECTION:
gmail.com. 3599 IN MX 20 alt2.gmail-smtp-in.l.google.com.

? dig -t txt gmail.com | grep -A1 "ANSWER SECTION"
;; ANSWER SECTION:
gmail.com. 299 IN TXT "v=spf1 redirect=_spf.google.com"
? dig -t txt _spf.google.com | grep -A1 "ANSWER SECTION"
;; ANSWER SECTION:
_spf.google.com. 299 IN TXT
    "v=spf1 include:_netblocks.google.com
```

```
include:_netblocks2.google.com  
include:_netblocks3.google.com ~all"
```

Otras características del DNS

- Transferencia de Zona: AXFR.
 - Entre servidores de DNS primario y secundario.
 - Se realiza sobre TCP de forma periódica.
- Dynamic DNS: DDNS.
 - Actualización dinámica de registros, usada con IP dinámicas.
- Split DNS.
 - Responder de acuerdo de donde proviene la consulta.

Cuestiones de Seguridad

- DNS no viaja cifrado ni autenticado.
- Para chequear integridad: DNSSEC (Desde auth ⇒ resolver).
- DNS over HTTPS (DoH) - Cambia como se usa la infraestructura de servidores.
- DNS over TLS (DoT) - Cambia como se usa la infra-estructura de servidores.
- DNS RPZ.

Clase 4

Clase 4

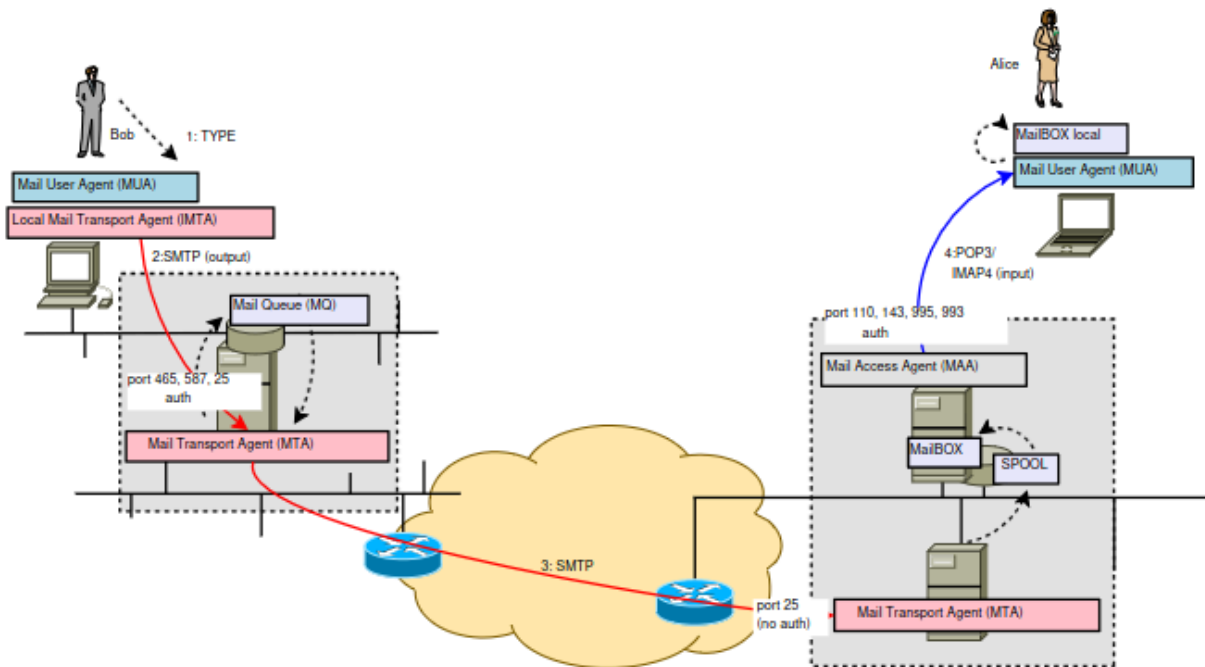
Mensajería de internet (E-mail)

Introducción

- Uno de los primeros servicios de Internet.
- Creado por Ray Tomlinson en 1971.
- Primeros protocolos de transporte UUCP (Unix-to-Unix Copy).
- Conexiones vía módems.
- Formato de E-mail: MACH2!MACH1!USER.
- En 1972: Se escoge el símbolo "@" para denotar "en", "at".

- Casilla de E-mail: USER@FQDN-DOMAIN o USER@FQDN-SERVER.
- En 1982: surge SMTP.
- En 1988: Gusano Morris, primer Virus, usaba el Sendmail.

Arquitectura simple

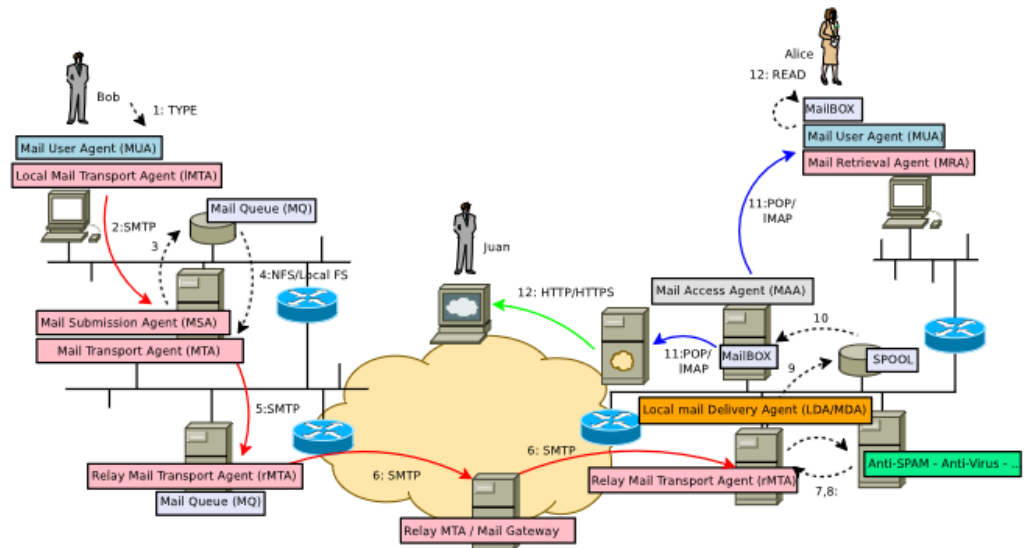


Arquitectura

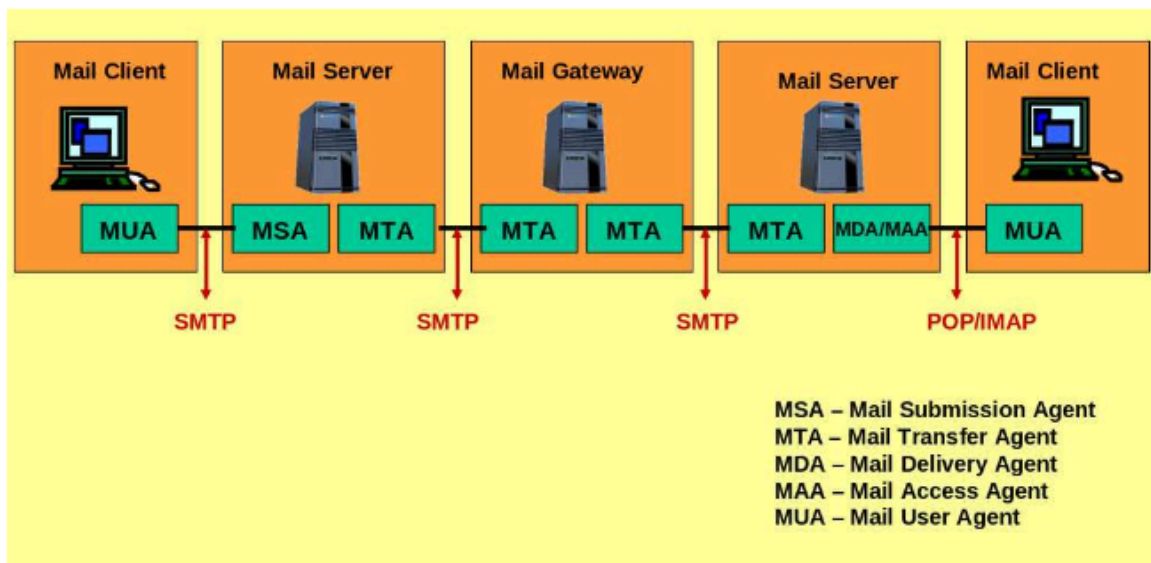
Arquitectura

- Componentes Principales:
 - MUA (Mail User Agent).
 - MTA (Mail Transport Agent).
 - MDA (Mail Delivery Agent).
- Componentes principales (desagregadas):
 - LDA (Local Delivery Agent), MDA local.
 - MAA (Mail Access Agent) (inc. en MDA).
 - MSA (Mail Submission Agent) (inc. en MTA).
 - MRA (Mail Retrieval Agent) (inc. en MUA)
- Componentes Secundarias:

- Srv. autenticación ext., Web Mail (Front-End WWW), srv. de Anti-Virus, Anti-SPAM, srv. de listas, ...



Componentes



fuelle: OPES WG - SMTP Use Cases at 62th. IETF (pres. paul.knight(at)nortel.com)

Arquitectura (MUA- Mail User Agent)

- Cliente.
- Interfaz con usuario.

- Lector/Editor/Emisor local de correos (e-mails).
- Posee integrado un local MTA para comunicarse con el Servidor de Mail Saliente, MTA que hace relay.
- El MTA que hace relay para el usuario pre-procesa el e-mail recibido desde el MUA con el agente MSA (Mail Submission Agent).
- Agrega la mayoría de los campos del header: Message-ID, To:, From:, Date:, Subject ...
- Posee integrado un MRA para comunicarse con el Servidor de Mail Entrante, MAA (Mail Access Agent).
- Utilizan protocolos SMTP o ESMTP, POP o IMAP, habla con MTA y con MAA propio. Ejemplos: Eudora, MS Outlook, Mozilla Thunderbird, elm, pine, mutt. Web-mailers: Horde/IMP, Squirrel, GMAIL, Yahoo, Hotmail

Arquitectura (MSA - Mail Submission Agent)

- Servidor.
- Agente habitualmente integrado en el MTA.
- Recibe el mensaje del MUA y lo pre-procesa antes de pasarlo al MTA para que haga el transporte.
- Agrega campos que pueden faltar, formato del header: Message-ID, To:, From:, Date: ...
- Termina de dar formato al header del e-mail.
- Utiliza protocolo SMTP, ESMTP.
- Habitualmente usaba el port 25, se recomienda usar 587(submission), RFC-6409, ex:RFC-4409.
- Ejemplos: componentes de Postfix (postdrop, pickup), Sendmail-MSA.

Arquitectura (MTA - Mail Transport Agent)

- Cliente y Servidor.
- Toma el e-mail desde el MSA o directamente desde el MUA (MSA integrado).

- Se encarga de enviar el mensaje de e-mail al servidor donde está la casilla de mensaje destino, comunicación de MTA a MTA.
- Almacena temporalmente el correo saliente.
- Se encarga de recibir y almacenar temporalmente los mensajes para las casillas que sirve desde el MTA remoto.
- Utiliza protocolo SMTP o ESMTP, entre servidores MTA.
- Ejemplos: Postfix, Sendmail, MS Exchange, Exim, Qmail.

Arquitectura (MD/LDA Mail Delivery Agent)

- Servicio interno o Servidor.
- Se encarga de tomar los e-mails recibidos por el MTA (acepta mensajes del MTA) y llevarlos al mailbox del usuario local.
- Se lo llama también LDA (Local Delivery Agent).
- Habitualmente define el formato del mailbox.
- Servicio de MDA puede hacer delivery remoto, cumple rol de MAA.
- Se integra con los protocolos POP y/o IMAP dejando los recursos disponibles al MAA o directamente al usuario.
- Ejemplos: procmail, postfix local, Sendmail, courier, cyrus-IMAP, dovecot, sieve, fetchmail(mas un MRA), getmail(más un MRA).

Arquitectura (MAA - Mail Access Agent)

- Servidor.
- Integrado o separado del MDA.
- Autentica el MUA/usuario y lee los e-mails del mailbox local dejados por el MDA/LDA.
- Transporta los e-mails hacia el MUA o los hace accesibles a este
- Se integra con el MDA. Implementa los protocolos POP y/o IMAP dejando acceder a los recursos y dialogando con el MUA.
- Ejemplos de MDA/LDA: courier, cyrus-IMAP, dovecot, sieve

SMTP

SMTP - Protocolo

- Simple Mail Transport Protocol.
- Protocolo Cliente/Servidor
- Utiliza formato ASCII 7 bits en 8 NVT.
- Usa TCP puerto servidor: 25.
- Los LMTA de los MUA hablan SMTP con su servidor SMTP saliente.
- Los servidores SMTP hablan entre sí este protocolo.
- RFC-821 (SMTP), extendida por RFC-1869 (ESMTP) y Redefinida por RFC-2821.

Ejemplo

```
? telnet 172.20.1.5 25
...
220 brasilia.cities.org SMTP Sendmail 8.12.8/8.12.8 ...
HELO server.other.test
250 brasilia.cities.test Hello server... pleased to meet
you
MAIL FROM: <bob@other.test>
250 2.1.0 bob@other.test... Sender ok
RCPT TO: <alice@cities.org>
250 2.1.5 alice@cities.org ... Recipient ok
DATA
354 Enter mail, end with "." on a line by itself
Esto es una prueba
.
250 2.0.0 k5HF1B0d005647 Message accepted for delivery
QUIT
221 2.0.0 brasilia.cities.org closing connection
```

-
- Requiere finalizar con CRLF, CRLF.
 - Usa conexiones persistentes.
 - Trabaja de forma Interactiva (Requerimiento/Respuesta) o Pipeline.
 - Puede o no requerir Autenticación.
 - Puede o no trabajar de forma segura: SSL/TLS.

Formato del mensaje

- El concepto de Envelope fue definido en RFC-821.
- Definido el cuerpo y el Encabezado en RFC-822, Redefinido RFC-2822.
 - Envelope (Envoltorio), el usuario no lo ve, usado por MTAs, MAIL FROM:, RCPT TO:
 - Headear (Encabezado), meta información del mail: Subject:, From:, To:, Return-Path: X-Mailer:, X-...
 - Body (Cuerpo), separada por línea en blanco del header:
Contenido del e-mail

Formato del mensaje Extendido

- Extensiones para enviar datos binarios.
- MIME (Multipurpose Internet Mail Extensions): Definido en RFC-1521 y RFC-1522. Redefinidas en RFC-2045 y RFC-2046.
- Con tags especiales indica el tipo al sistema destino.
- Luego codifica el mensaje binario en formato que no viole US-ASCII (NVT).
- Algunos ejemplos: uunecode, base64, quotes-strings, etc.

Formato del mensaje (ejemplo)

```
From : bob@other . test
To : alice@cities . org
Subject : Test Mail MIME
Date : Thu , 11 Aug 2005 20:20:56 -0300 ( ART )
MIME - Version : 1.0
Content - Transfer - Encoding : base64
Content - Type : image / jpeg
base64 encoded image .....
.....
..... base64 encoded image
```

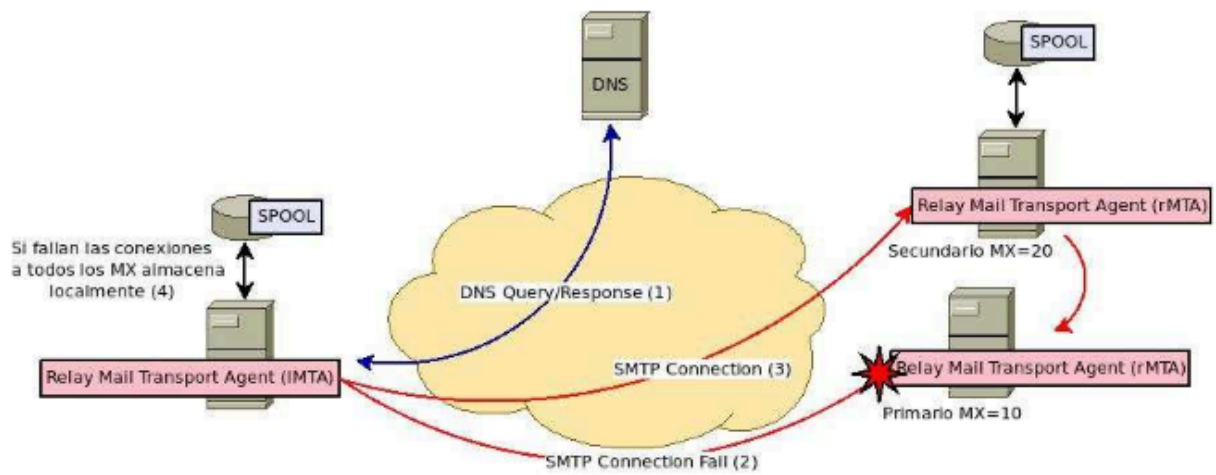
Mensajes Multipart (ejemplo)

```
From : bob@other . test
To : alice@cities . org
Subject : Test Mail MIME Multipart
Date : Thu , 11 Aug 2005 20:20:56 -0300 ( ART )
MIME - version : 1.0
Content - type : multipart / mixed ; boundary =" D D D I I
I V V V I I I SS SO OO RRR"
This is a message with multiple parts in MIME format .
-- D D D I I I V V V I I IS SS 000 RR R
Content - type : text / plain
Mensaje en Texto
-- D D D I I I V V V I I IS SS 000 RR R
Content - type : application / octet - stream
Content - transfer - encoding : base64
base64 encoded stream .....
. . . . .
..... base64 encoded stream
-- DDIIIIVVVIIIISSSO00RRR - -
```

Protocolo ESMTP (SMTP Extendido)

```
220 brasilia.cities.org ESMTP Sendmail 8.12.8/8.12.8 ...
EHLO server.other.test
250 brasilia.cities.test Hello server..., pleased ...
250-ENHANCEDSTATUSCODES
250-PIPELINING
250-STARTTLS
250-AUTH
250-8BITMIME
250-SIZE
250-DSN
250-ETRN
250-VRFY
250-EXPN
250 HELP
```

SMTP y DNS



Protocolos Acceso a Correo

- POP: Post Office Protocol, RFC-1939: POPv3.
- IMAP: Internet Mail Access Protocol, RFC-1730: IMAPv4.
- Requieren Autenticación.
- Utiliza formato ASCII 7 bits en 8 NVT.
- Usan TCP puertos servidor: 110 y 143.
- Permiten correr de forma segura sobre SSL/TLS.
- IMAP más flexible permite uso de carpetas y manipulación de mensajes en el servidor.

POP (Ejemplo)

```
? telnet 127.0.0.1 110
...
+OK POP3 localhost.localdomain v2001.78rh server ready
user alice
+OK User name accepted, password please
pass 123456
+OK Mailbox open, 2 messages
NOEXIST
-ERR Unknown TRANSACTION state command
LIST
+OK Mailbox scan listing follows
1 620
```

2 622

TOP 1 100

+OK Top of message follows

Return-Path: <juan@...>

...

TOP 2 100

+OK Top of message follows

Return-Path: <root@...>

...

QUIT

+OK Sayonara

IMAP (ejemplo)

? telnet 127.0.0.1 143

...

* OK brasilia.cities.org IMAP4rev1 v12.264 server ready

A0001 USER "alice" "123456"

* OK User alice authenticated

A0002 SELECT INBOX

* 1 EXISTS

* 1 RECENT ...

A0004 FETCH 1 BODY[HEADER] ### Get first message header

...

A0005 FETCH 1 BODY[TEXT] ### Get first message body

...

A0005 OK FETCH completed

A0006 LOGOUT

* BYE example.com IMAP4rev1 server terminating connection

A0006 OK LOGOUT completed

Spam

- Open Relay.
- SPAMbots.

- Métodos para Contrarrestarlo.
 - RBL (Realtime Blackhole List) Blacklist/Denylist.
 - Greylist, Whitelist/Allowlist.
 - Filtros de Contenido.
 - Reglas Dirigidas/Entrenadas por el usuario.
 - SPF (Sender Policy Framework) , DKIM (DomainKeys Identified Mail).